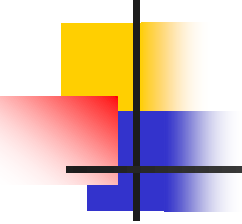


Padrões de Projeto

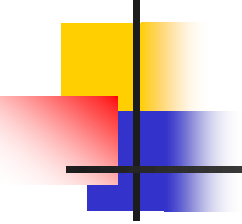
Prof^a. Rachel Reis
rachel@inf.ufpr.br



Padrões – Factory Method

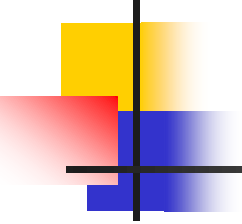
- Exemplos:

Criação	Estrutural	Comportamental
• Abstract factory • Builder • Factory Method • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Façade • Flyweight • Proxy	• Chain of responsibility • Command • Interpreter • Iterator • Mediator • Memento • Observer • Etc.



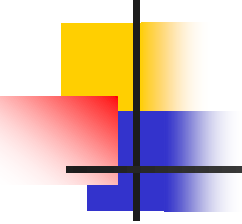
Sobre o Factory Method

- É um padrão de projeto de criação (lida com a criação de objetos)
- Oculta a lógica de instanciação do código cliente (desacopla o código que cria o objeto do código que utiliza o objeto).
- Utiliza os conceitos de interface, classe abstrata e herança.



Sobre o Factory Method

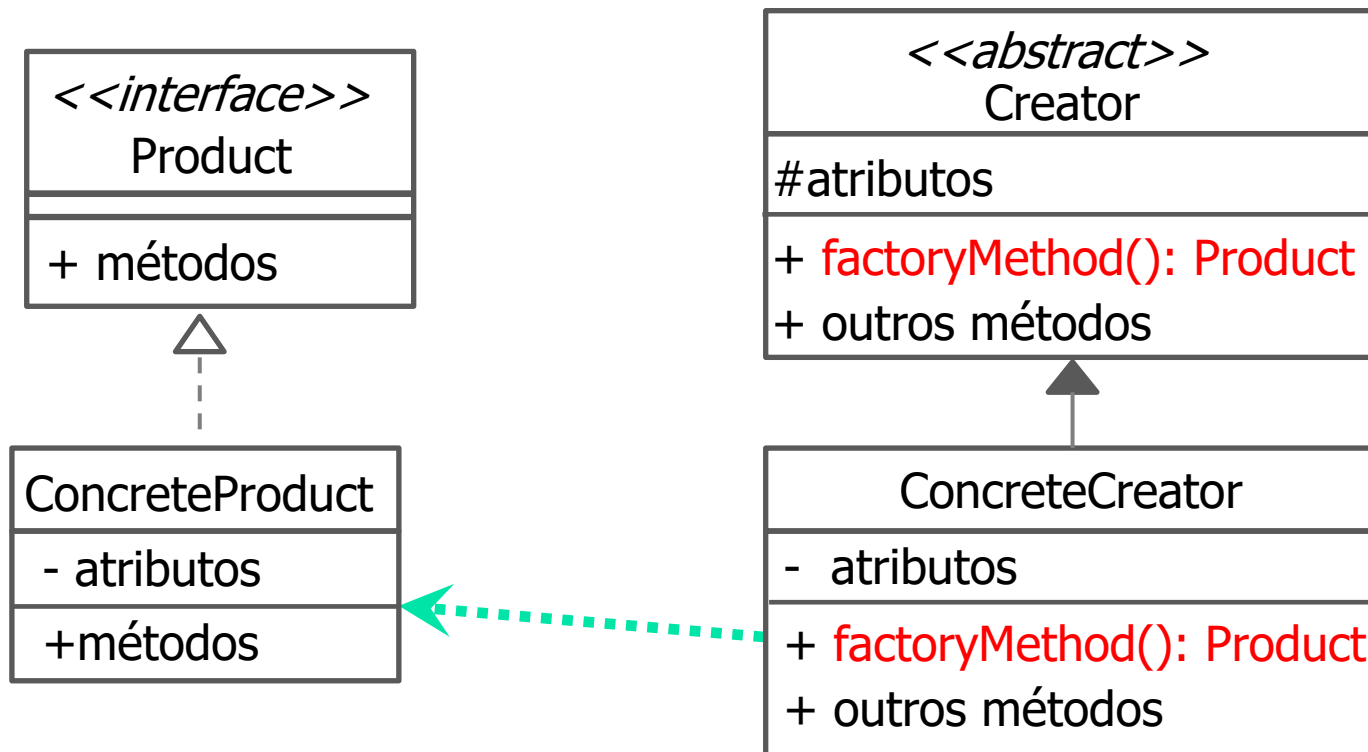
- Oferece flexibilidade ao código permitindo a criação de novas *factories* sem a necessidade de alterar o código já escrito.
- Pode usar parâmetros para determinar o tipo dos objetos a serem criados ou receber esses parâmetros para repassar aos objetos que estão sendo criados.

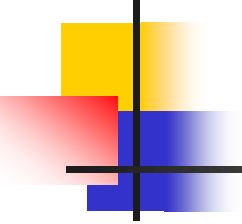


Factory Method - Intenção

- Definir uma interface para criar um objeto, mas deixar as subclasses decidirem que classe instanciar.
- Permite a uma classe adiar a instanciação para as subclasses.

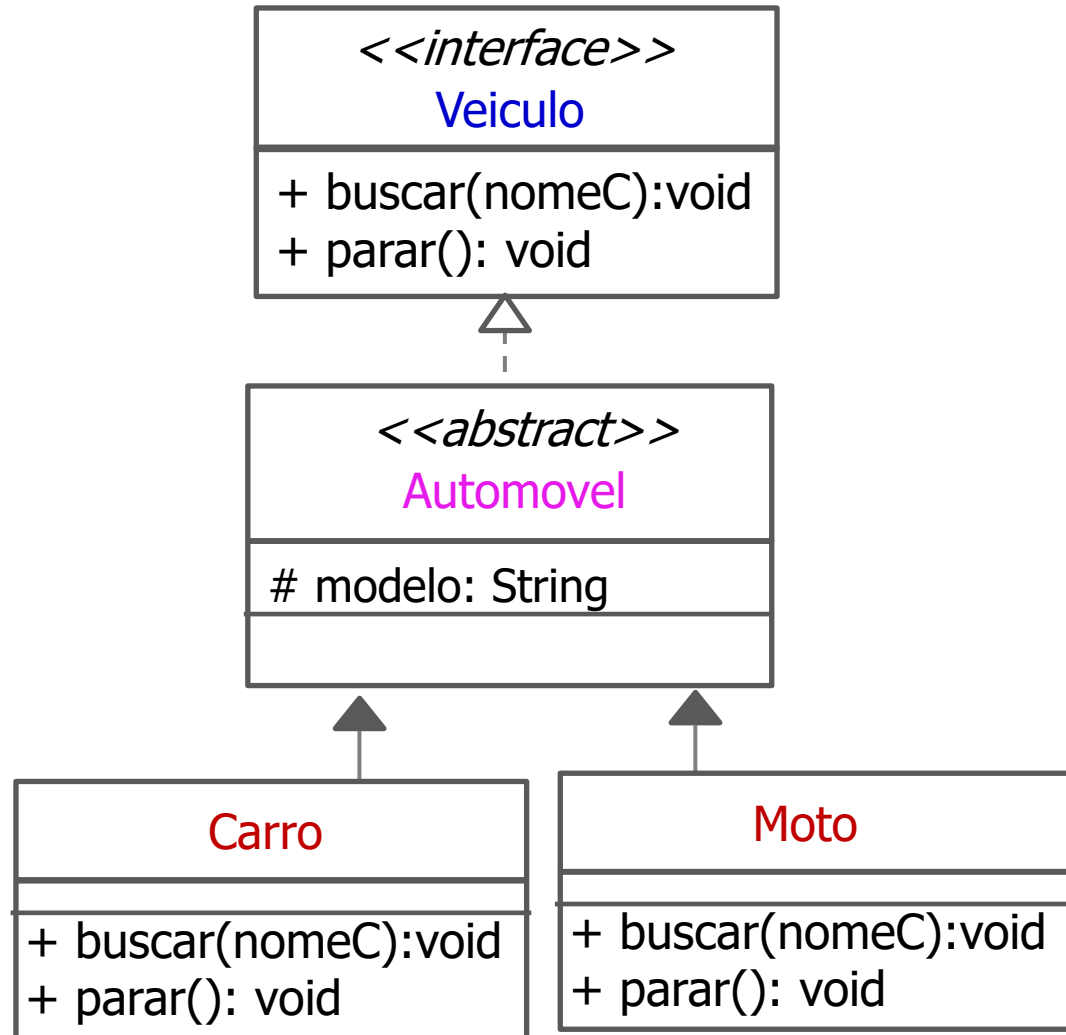
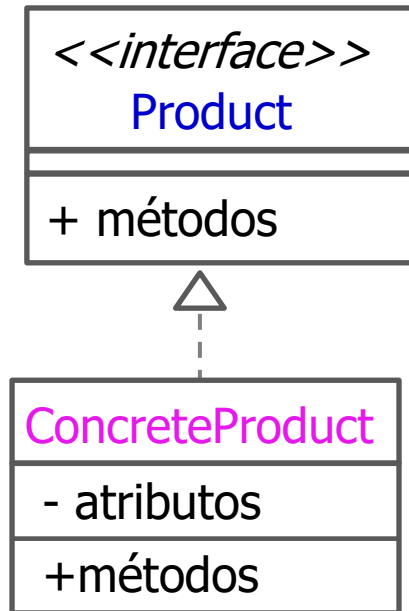
Factory Method - Estrutura



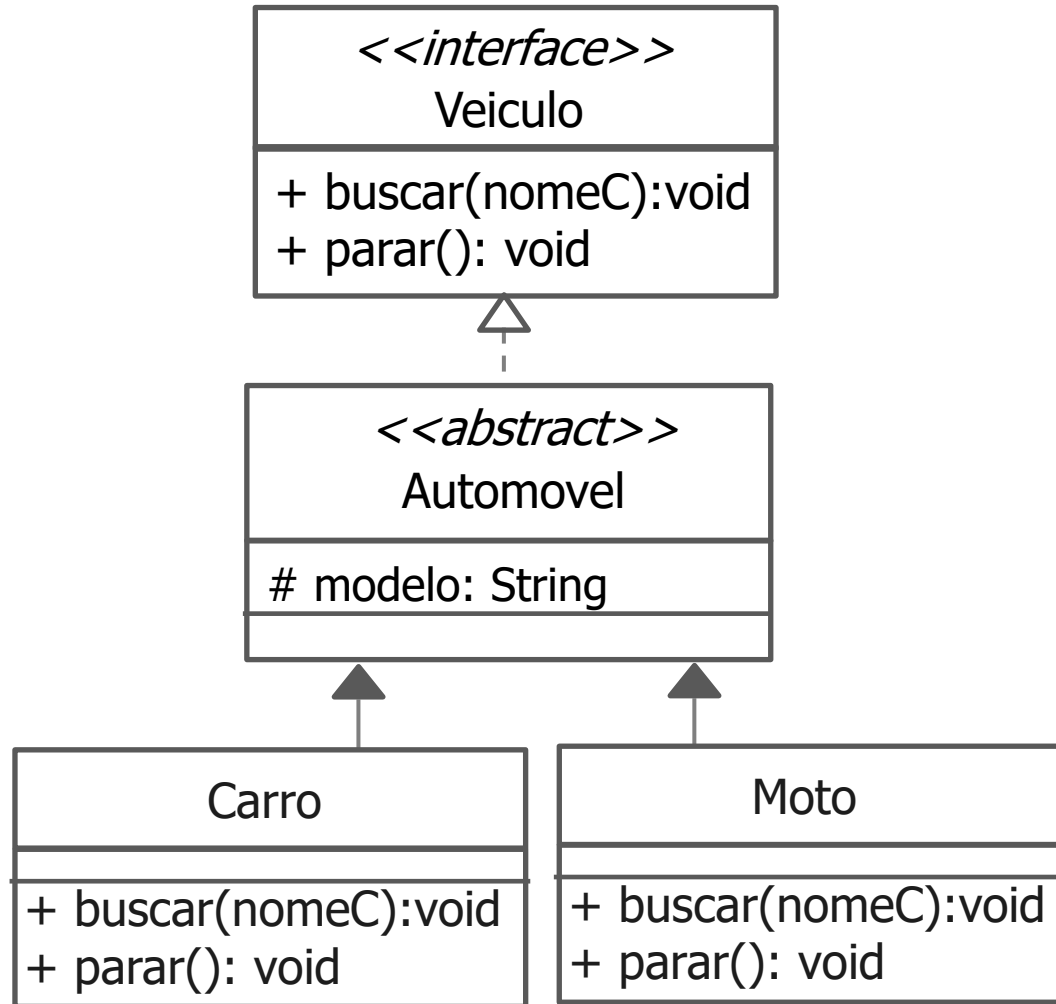


Exemplo 1: Aplicação Uber

- Desenvolva uma aplicação Uber que ofereça o serviço de transporte de pessoas (similar ao taxi) em carros e motos.



Implementar a estrutura abaixo



```
public interface Veiculo
{
    public abstract void buscar(String nomeC);
    public abstract void parar();
}
```

Veiculo.java

<<interface>>

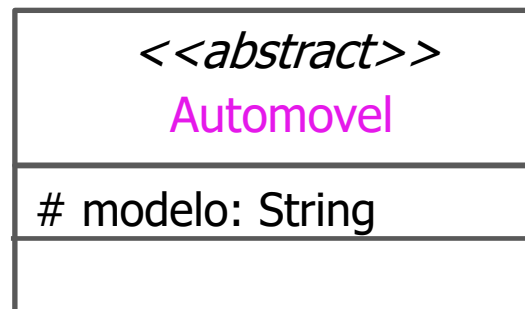
Veiculo

+ buscar(nomeC):void

+ parar(): void

```
public abstract class Automovel implements Veiculo{  
    protected String modelo;  
  
    public Automovel(String modelo){  
        this.setModelo(modelo);  
    }  
  
    // Implementar métodos get/set  
  
}
```

Automovel.java



```
public class Carro extends Automovel  
{
```

```
    public Carro(String modelo){  
        super(modelo);  
    }
```

```
    public void buscar(String nomeC){  
        System.out.println(this.modelo + " buscando " + nomeC);  
    }  
    public void parar(){  
        System.out.println(this.modelo + " parado");  
    }  
}
```

<<abstract>>

Automovel



Carro

+buscar(nomeC):void
+parar(): void

Carro.java

```
public class Moto extends Automovel  
{
```

```
    public Moto(String modelo){  
        super(modelo);  
    }
```

```
    public void buscar(String nomeC){  
        System.out.println(this.modelo + " buscando " + nomeC);  
    }  
    public void parar(){  
        System.out.println(this.modelo + " parada");  
    }  
}
```

```
}
```

<<abstract>>
Automovel



Moto

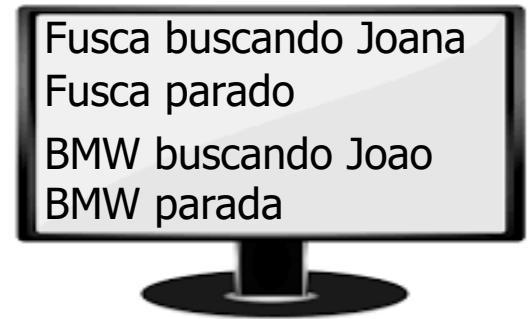
+buscar(nomeC):void
+parar(): void

Moto.java

```
public class Principal
{
    public static void main(String []args)
    {
        // Código sem usar o padrão Method Factory
        Veiculo fusca = new Carro("Fusca");
        fusca.buscar("Joana");
        fusca.parar();

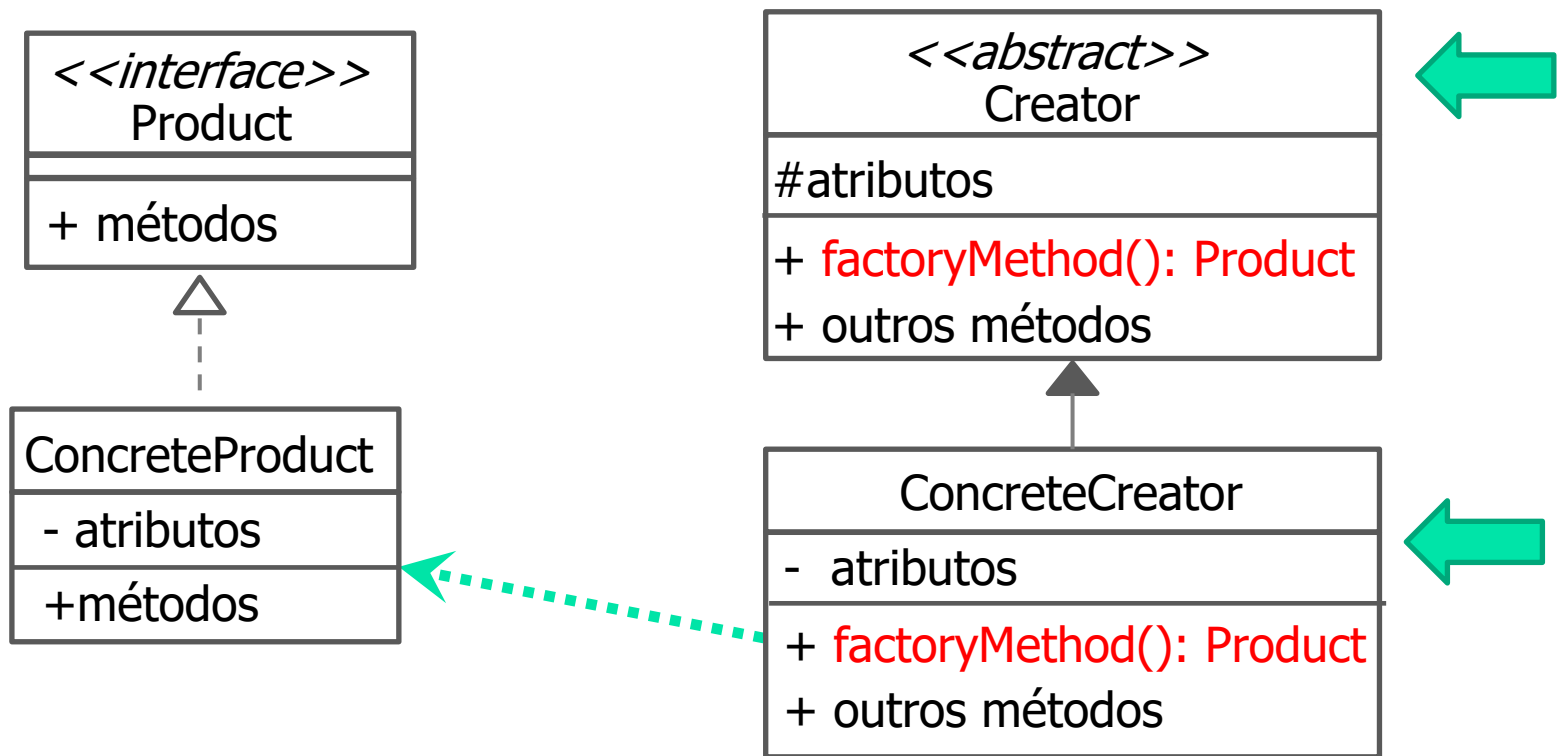
        Veiculo honda = new Moto("BMW");
        honda.buscar("João");
        honda.parar();

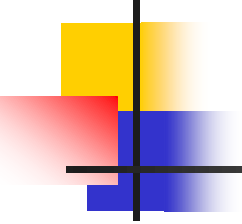
    }
}
```



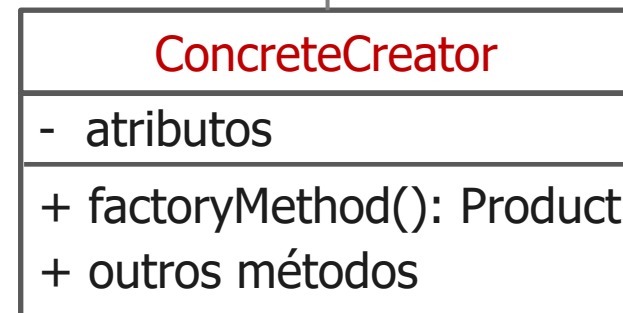
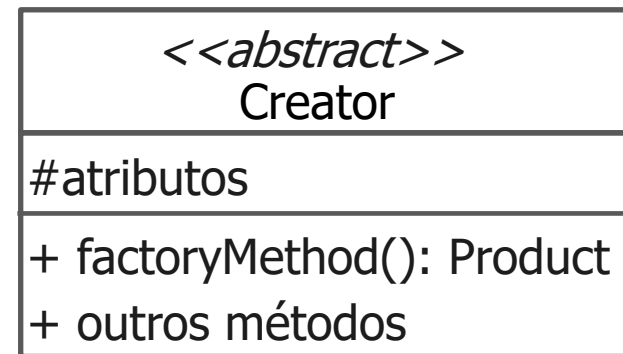
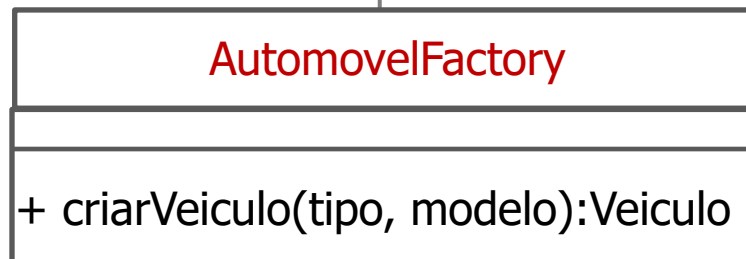
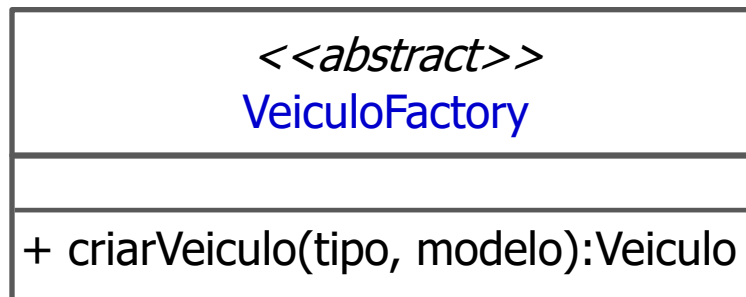
O que acontece se alterarmos o nome das classes Carro e Moto para Carro1 e Moto1?

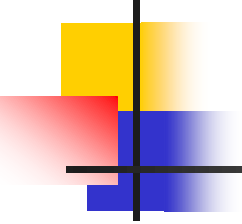
- Para solucionar o problema anterior vamos usar o padrão de projeto Method Factory



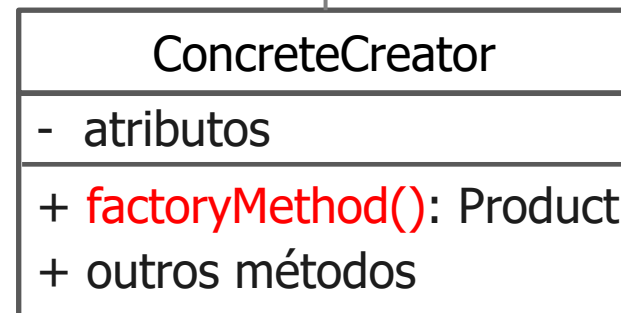
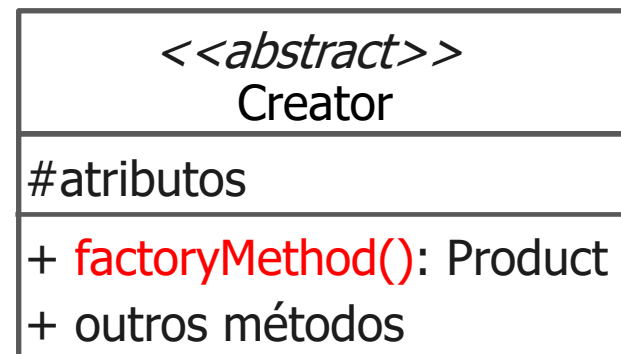
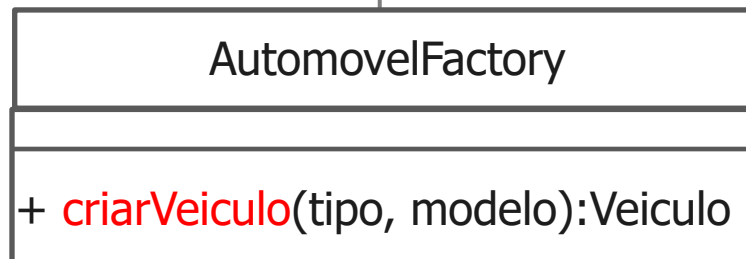
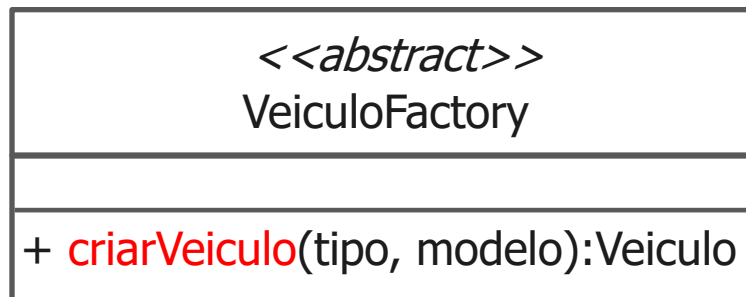


Exemplo: Aplicação - Uber

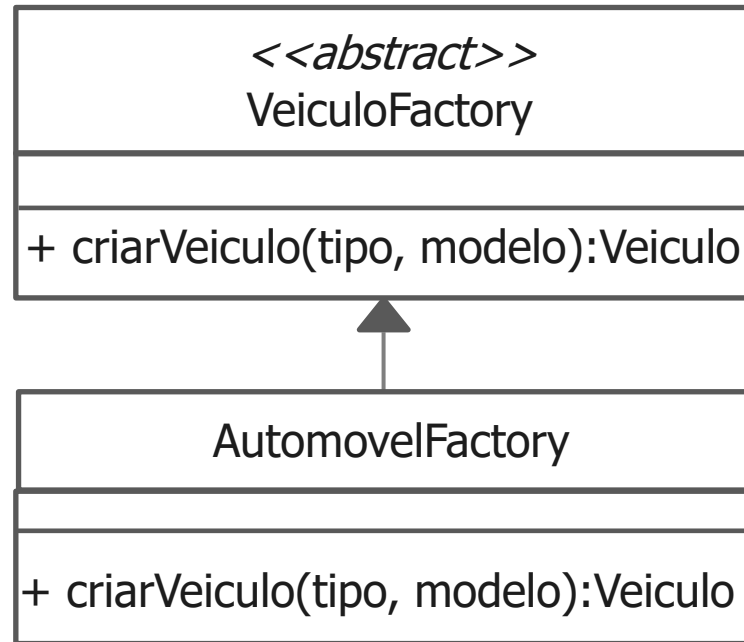




Exemplo: Aplicação - Uber

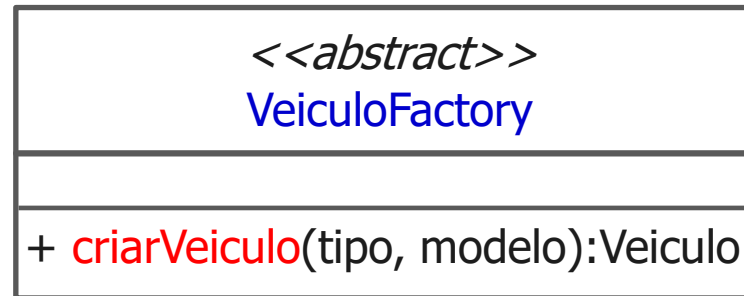


Implementar a estrutura abaixo



```
public abstract class VeiculoFactory
{
    public abstract Veiculo criarVeiculo (String tipo, String modelo);
}
```

VeiculoFactory.java



```
public class AutomovelFactory extends VeiculoFactory
{
    public Veiculo criarVeiculo (String tipo, String modelo)
    {

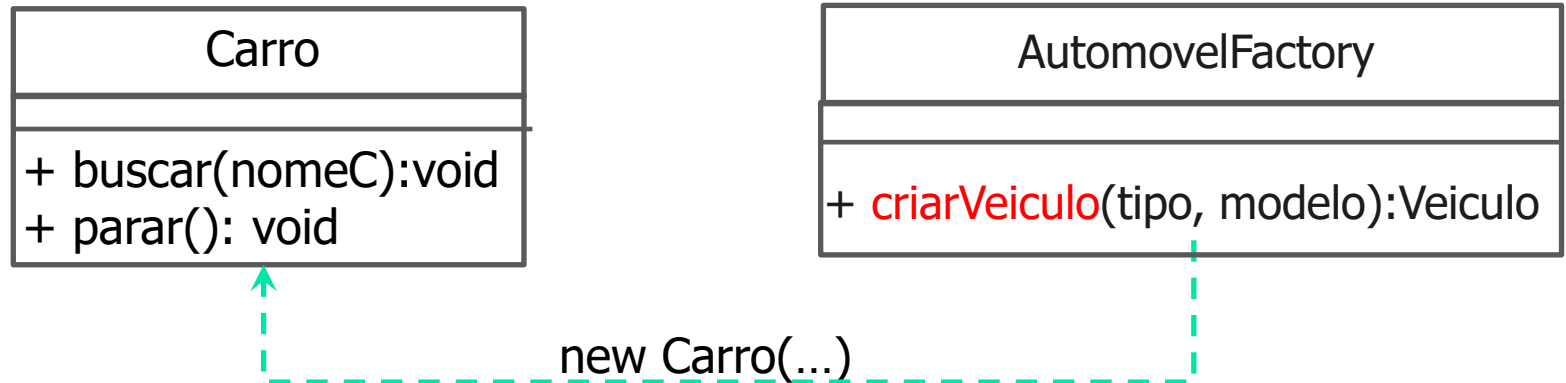
    }
}
```

AutomovelFactory.java

AutomovelFactory

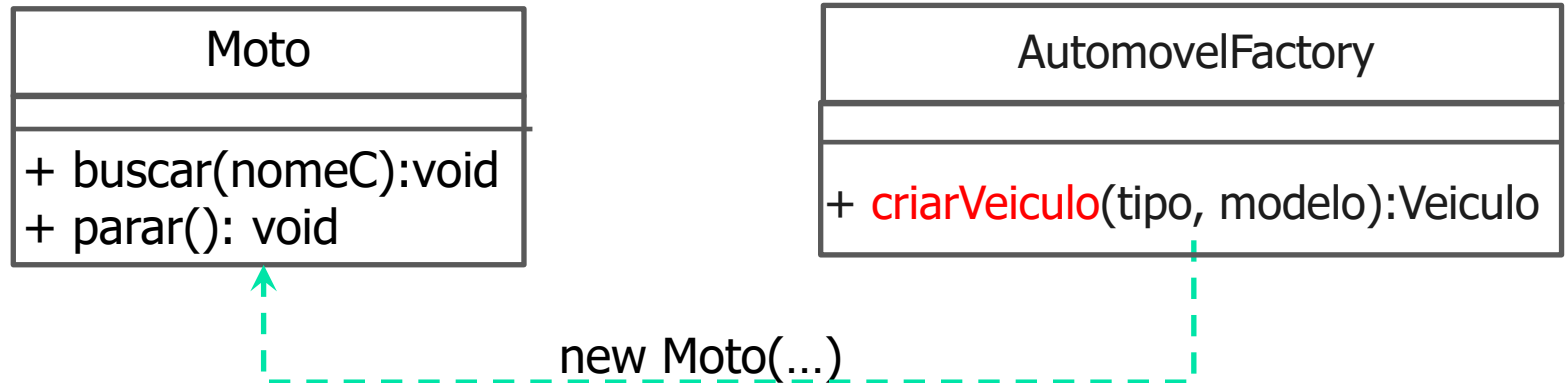
+ criarVeiculo(tipo, modelo):Veiculo

```
public class AutomovelFactory extends VeiculoFactory
{
    public Veiculo criarVeiculo (String tipo, String modelo){
        ➡ if(tipo == "carro")
            return new Carro(modelo);
    }
}
```



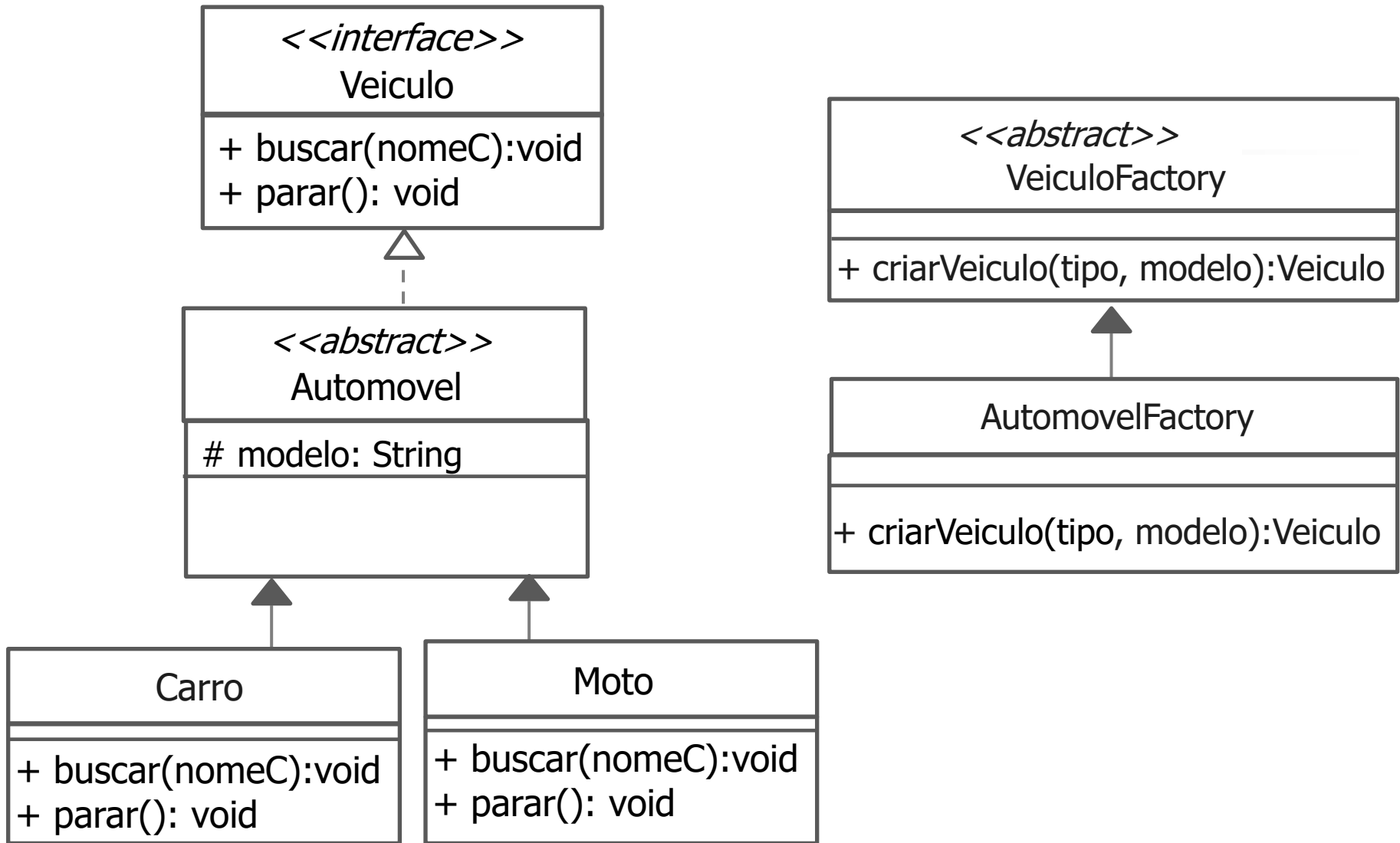
```
public class AutomovelFactory extends VeiculoFactory
{
    public Veiculo criarVeiculo (String tipo, String modelo){
        if(tipo == "carro")
            return new Carro(modelo);
        ➡ else if(tipo == "moto")
            return new Moto(modelo);

    }
}
```



```
public class AutomovelFactory extends VeiculoFactory
{
    public Veiculo criarVeiculo (String tipo, String modelo){
        if(tipo == "carro")
            return new Carro(modelo);
        else if(tipo == "moto")
            return new Moto(modelo);
         else
            return null;
        }
    }
}
```

Estrutura completa: Aplicação - Uber




```
public class Principal
```

```
{
```

```
    public static void main(String []args)
```

```
    {
```

```
        // Código usando o padrão Method Factory
```

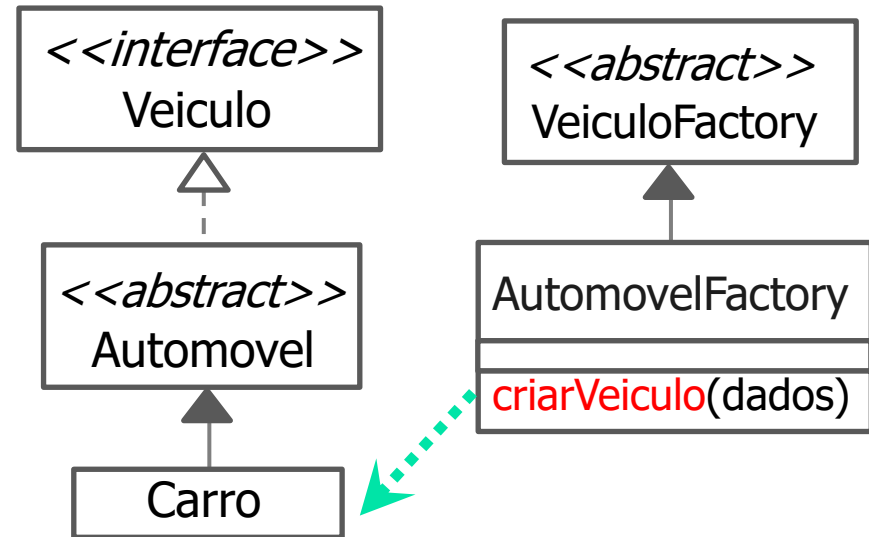
```
        VeiculoFactory autoF = new AutomovelFactory();
```

```
        Veiculo v = autoF.criarVeiculo("carro", "Fusca");
```

```
    }
```

```
}
```

```
}
```



```
public class Principal
```

```
{
```

```
    public static void main(String []args)
```

```
    {
```

```
        // Código usando o padrão Method Factory
```

```
        VeiculoFactory autoF = new AutomovelFactory();
```

```
        Veiculo v = autoF.criarVeiculo("carro", "Fusca");
```

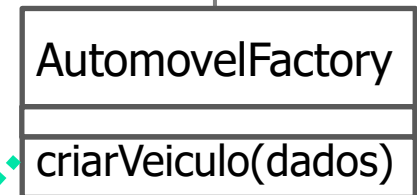
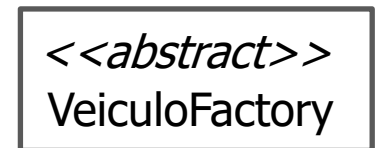
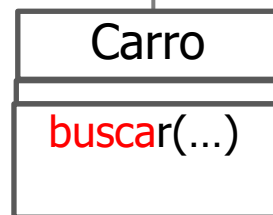
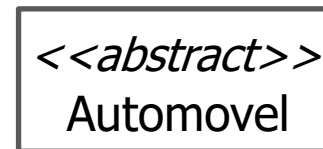
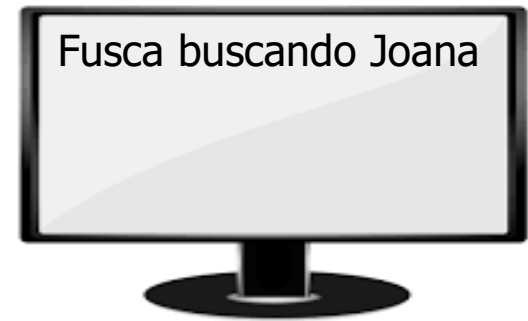
```
        if (v != null){
```

```
            v.buscar("Joana");
```

```
        }
```

```
    }
```

```
}
```



```
public class Principal
```

```
{
```

```
    public static void main(String []args)
```

```
    {
```

```
        // Código usando o padrão Method Factory
```

```
        VeiculoFactory autoF = new AutomovelFactory();
```

```
        Veiculo v = autoF.criarVeiculo("carro", "Fusca");
```

```
        if (v != null){
```

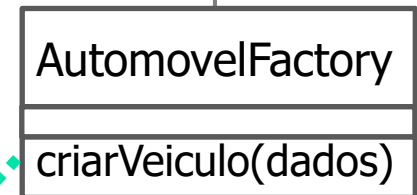
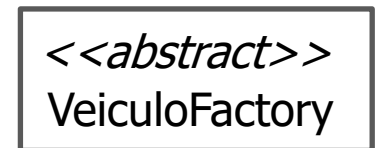
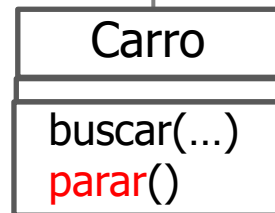
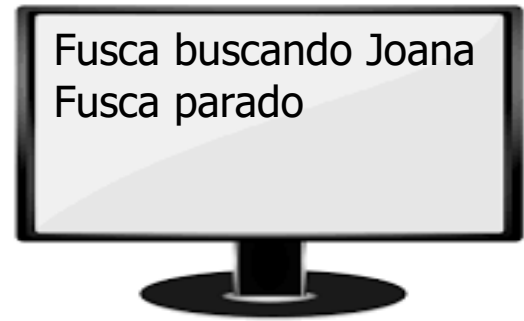
```
            v.buscar("Joana");
```

```
            v.parar();
```

```
        }
```

```
    }
```

```
}
```



```
public class Principal
{
    public static void main(String []args)
    {
        // Código usando o padrão Method Factory
        VeiculoFactory autoF = new AutomovelFactory();
        Veiculo v = autoF.criarVeiculo("carro", "Fusca");

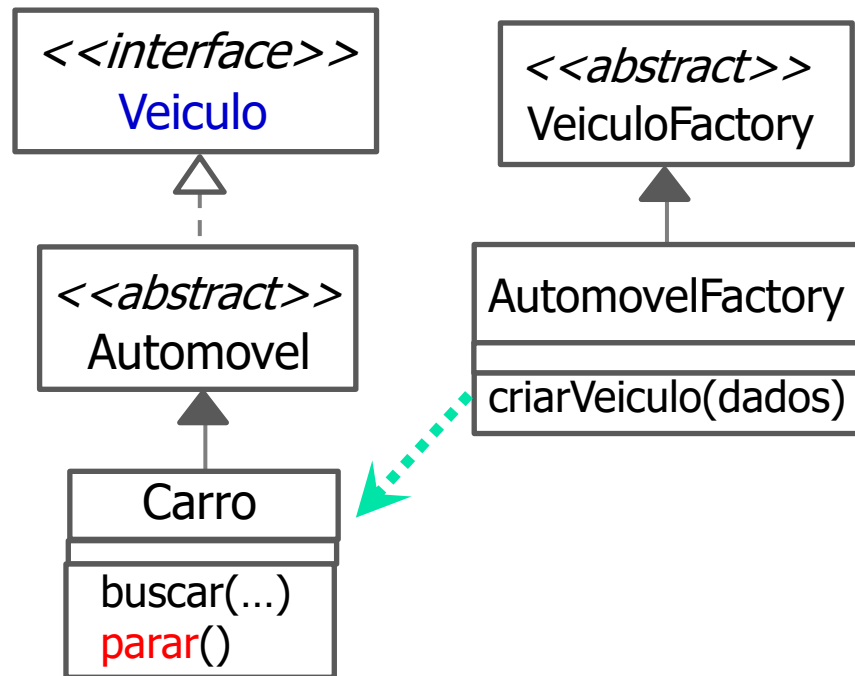
        if (v != null){
            v.buscar("Joana");
            v.parar();
        }
    }
}
```

Altere o código para que uma
moto BMW busque João

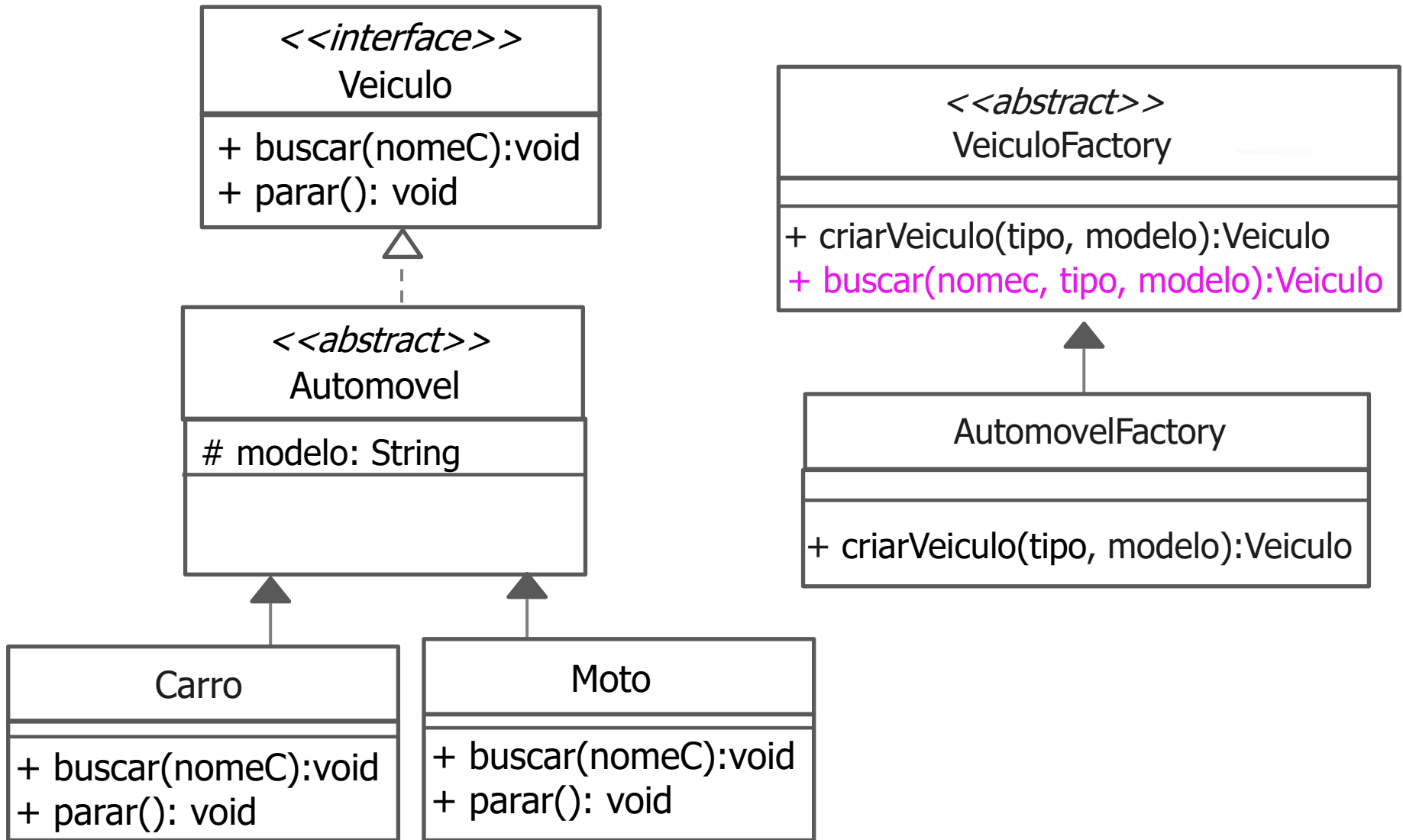
```
public class Principal{  
    public static void main(String []args)  
    {  
        // Código sem usar o padrão Method Factory  
        Veiculo fusca = new Carro("Fusca");  
        fusca.buscar("Joana");  
        fusca.parar();  
  
        // Código usando o padrão Method Factory  
        VeiculoFactory autoF = new AutomovelFactory();  
        Veiculo v = autoF.criarVeiculo("carro", "Fusca");  
        v.buscar("Joana");  
        v.parar();  
    }  
}
```

- Revendo a **intenção**: definir uma interface para criar um objeto, mas deixar as subclasses decidirem que classe instanciar.
- Permite a uma classe adiar a instanciação para as subclasses.

```
VeiculoFactory autoF = new AutomovelFactory();  
Veiculo v = autoF.criarVeiculo("carro", "Fusca");
```



- Podemos ter outros métodos na classe VeiculoFactory



```
public abstract class VeiculoFactory
{
    public abstract Veiculo criarVeiculo (String tipo, String modelo);

    public Veiculo buscar(String nomeC, String tipo, String modelo)
    {
        Veiculo v = this.criarVeiculo(tipo, modelo);
        v.buscar(nomeC);
        return v;
    }
}
```

<<abstract>>

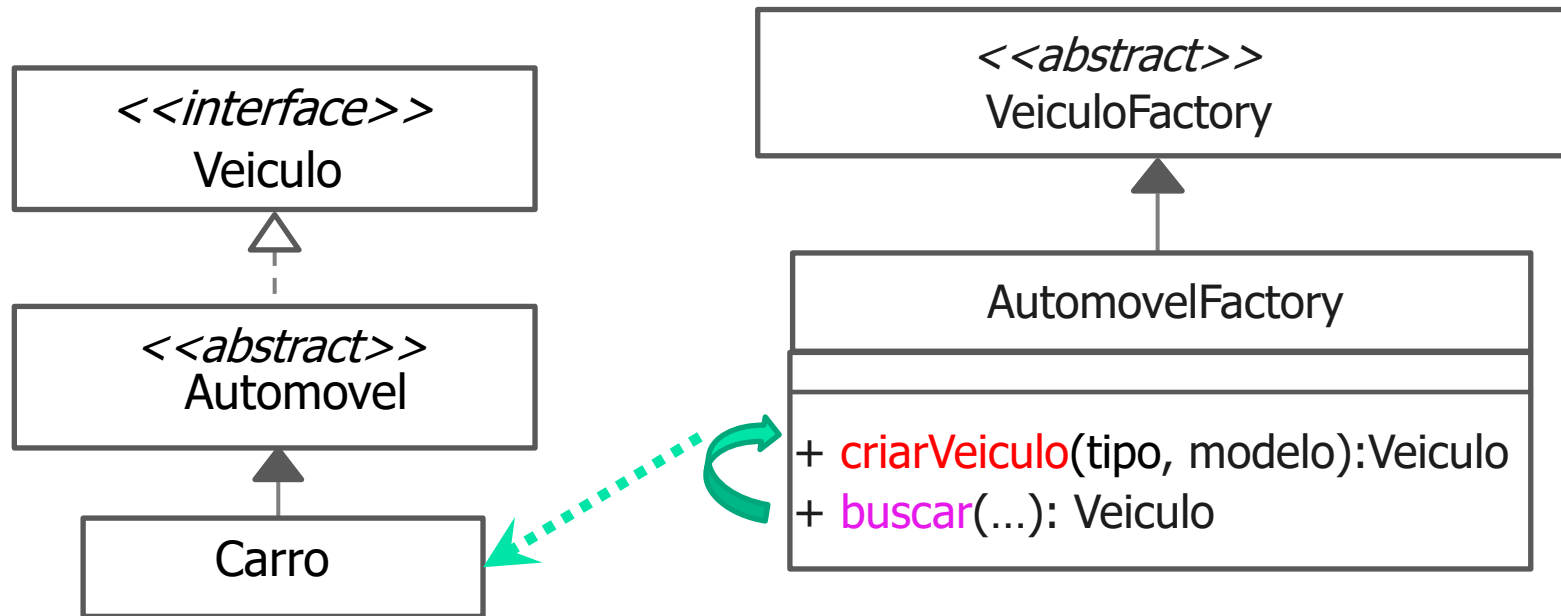
VeiculoFactory

+ criarVeiculo(tipo, modelo):Veiculo
+ buscar(nomeC, tipo, modelo):Veiculo

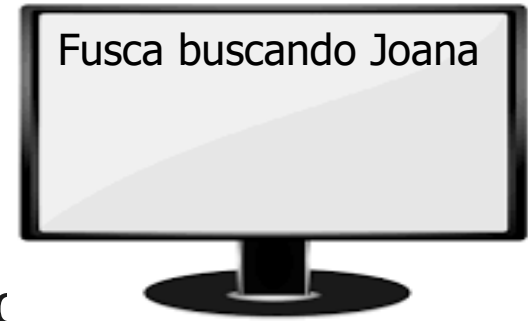

```

public class Principal{
    public static void main(String []args)
    {
        // Código usando o padrão Method Factory
        VeiculoFactory autoF1 = new AutomovelFactory();
        Veiculo v1 = autoF1.buscar("Joana", "carro", "Fusca");
    }
}

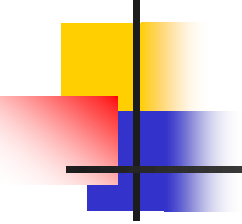
```



```
public class Principal{  
    public static void main(String []args)  
    {  
        // Código usando o padrão Method Factory  
        VeiculoFactory autoF1 = new AutomovelFacto  
        Veiculo v1 = autoF1.buscar("Joana", "carro", "Fusca");  
    }  
}
```



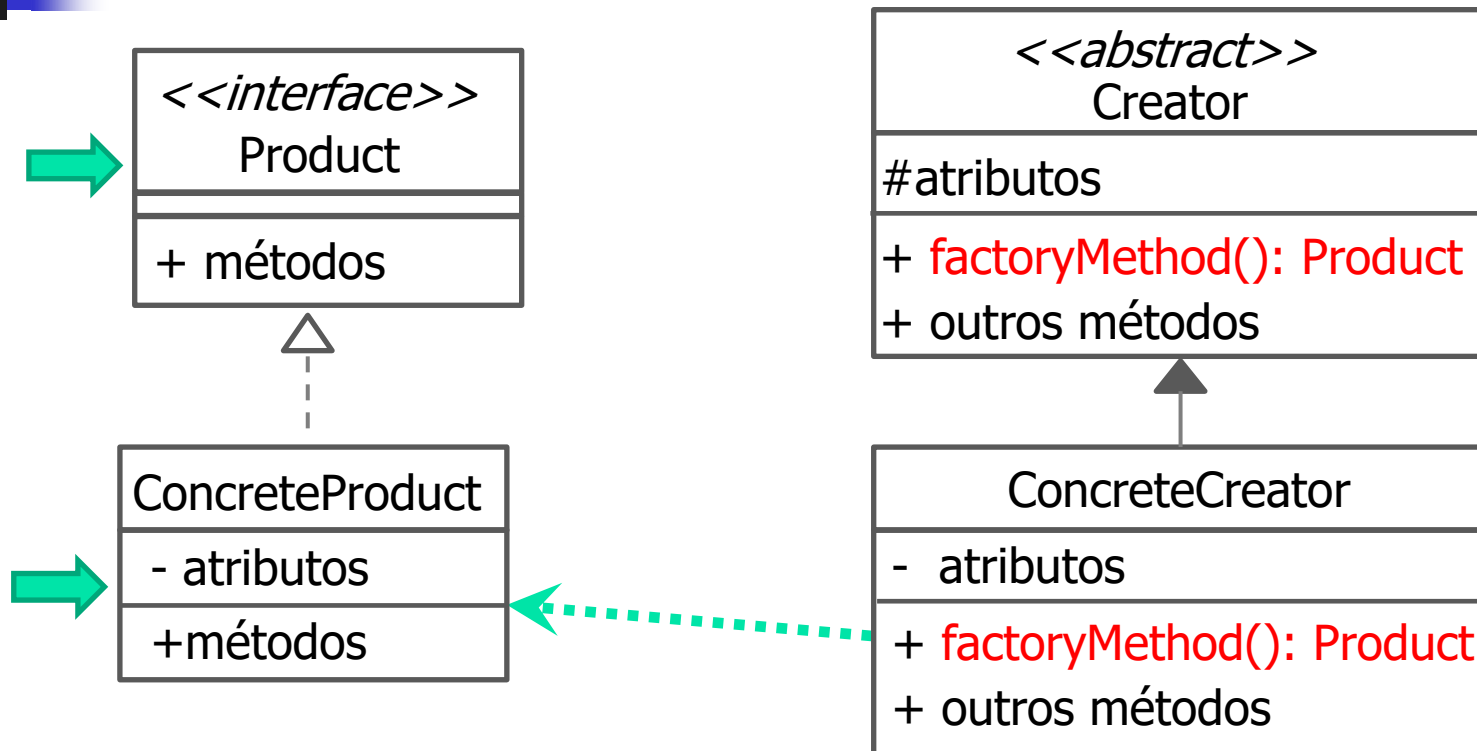
Como fazer para chamar
o método parar()?



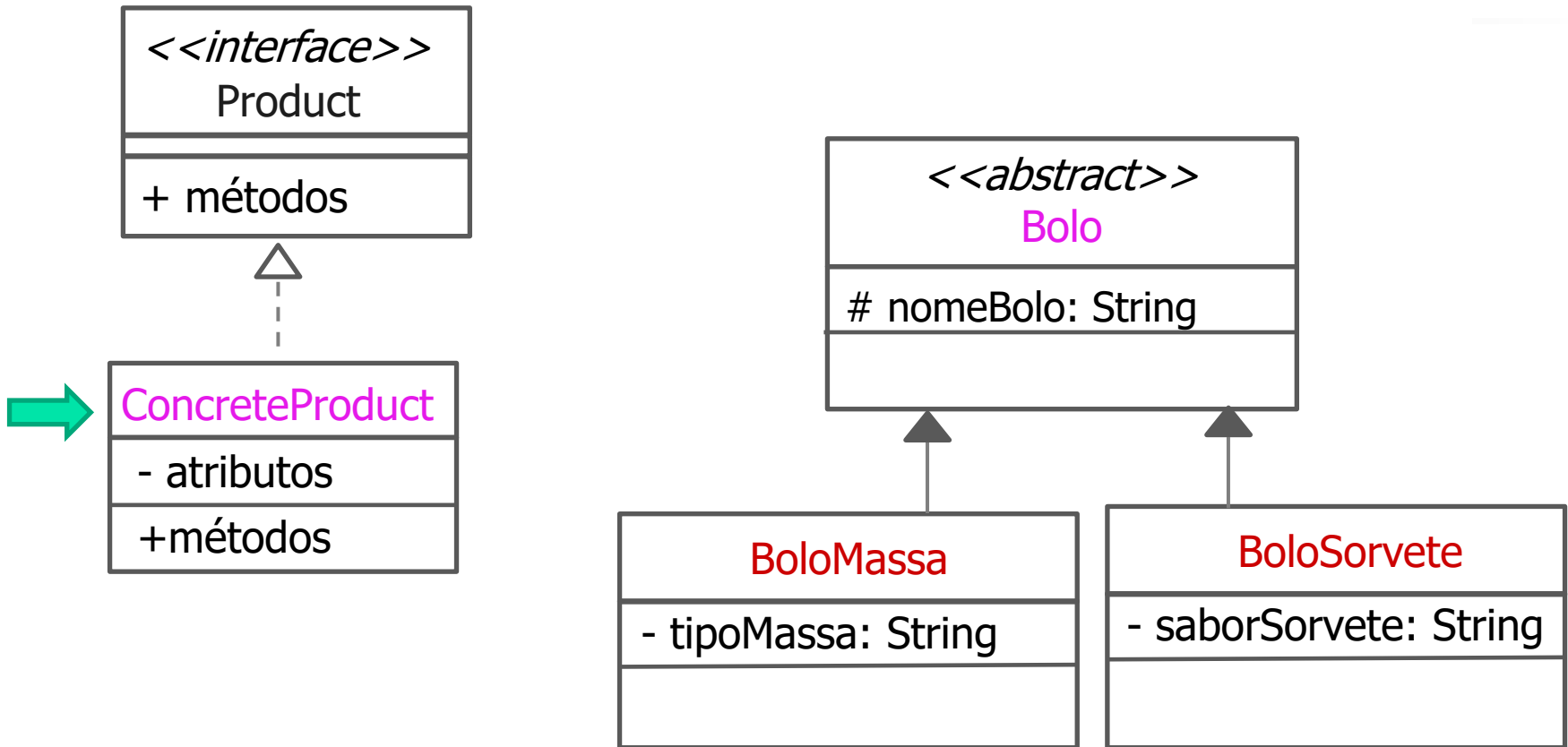
Exemplo 2: Aplicação de Receitas

- Desenvolva uma aplicação para uma empresa com o objetivo de produzir diferentes tipos de bolos.

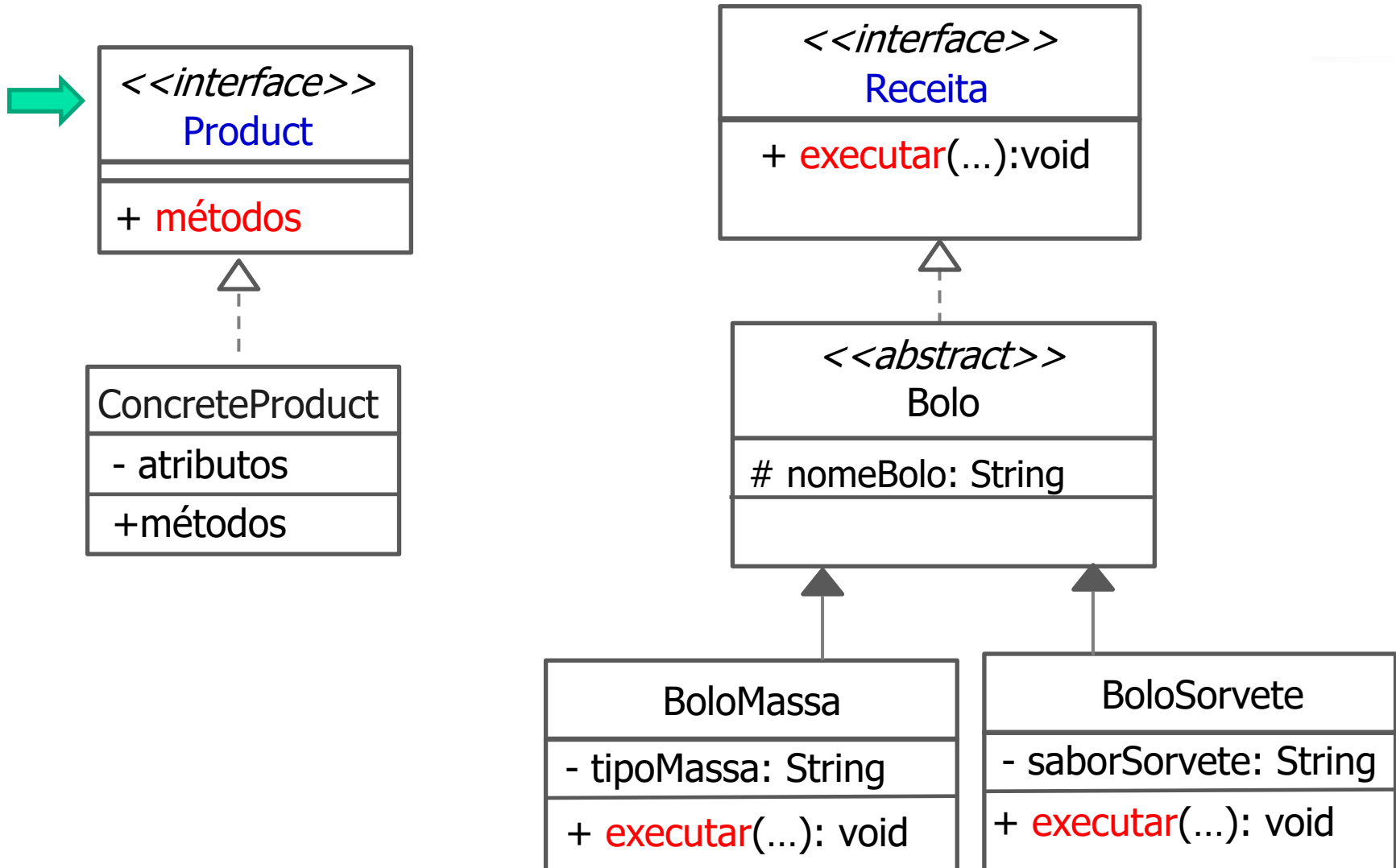
Factory Method - Estrutura



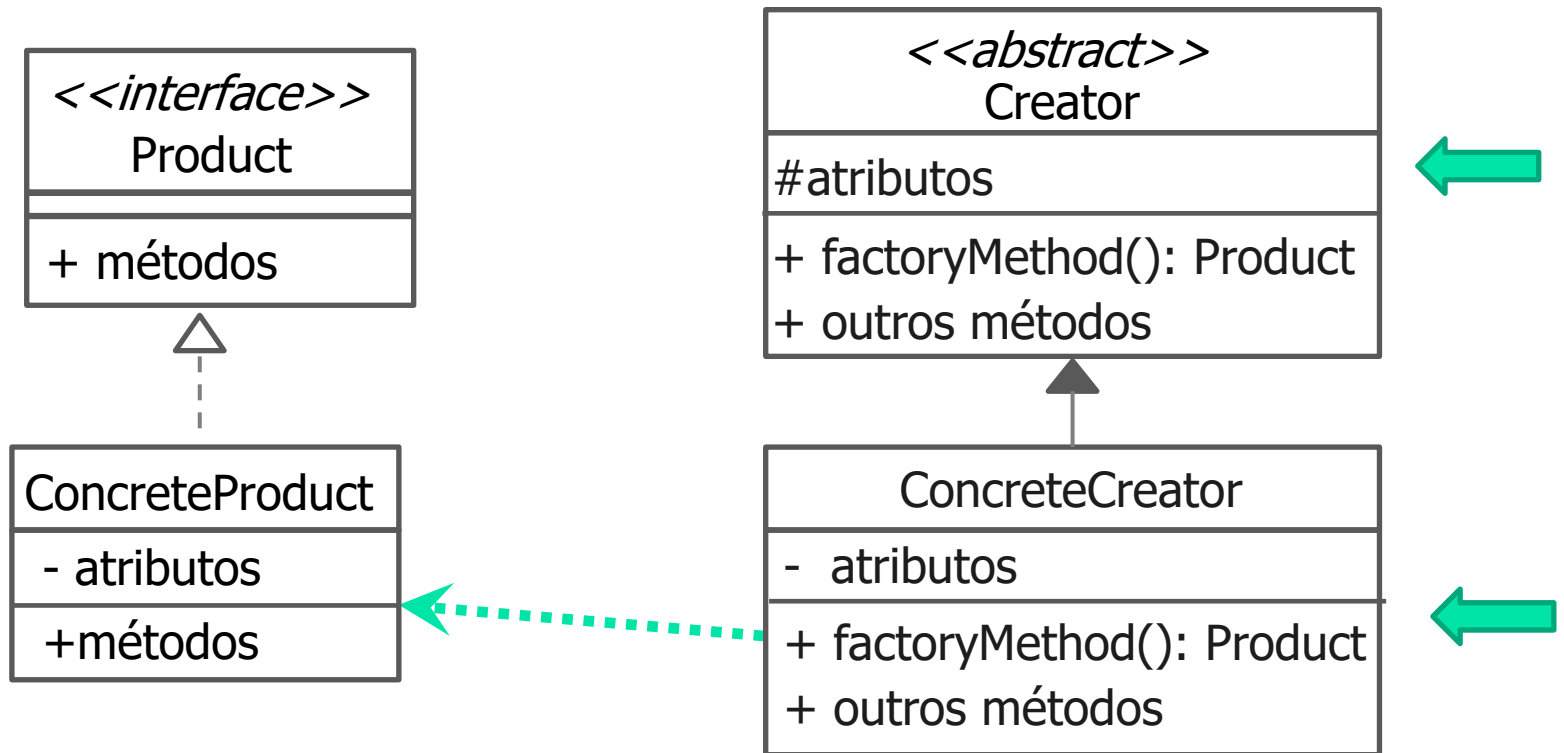
Quem vai ser a classe ConcreteProduct?



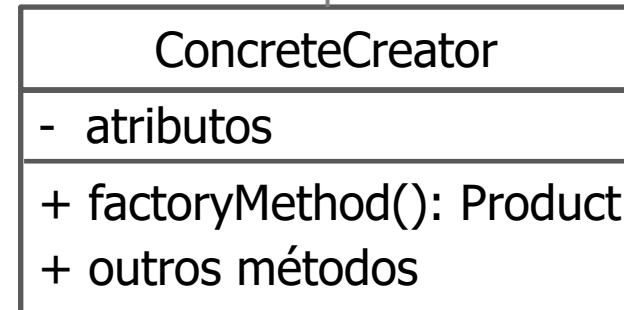
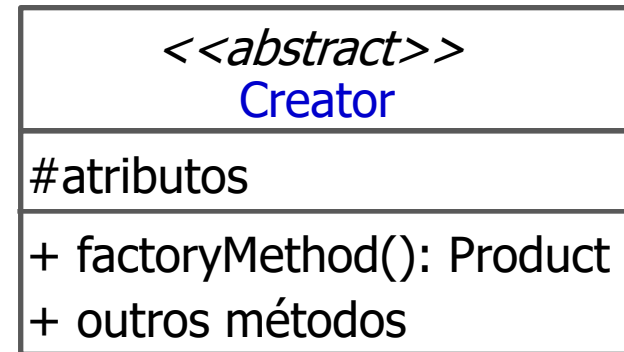
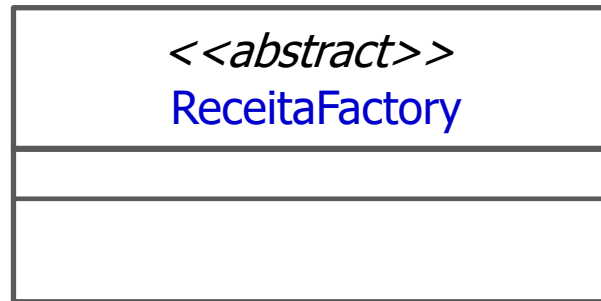
E a interface Product?



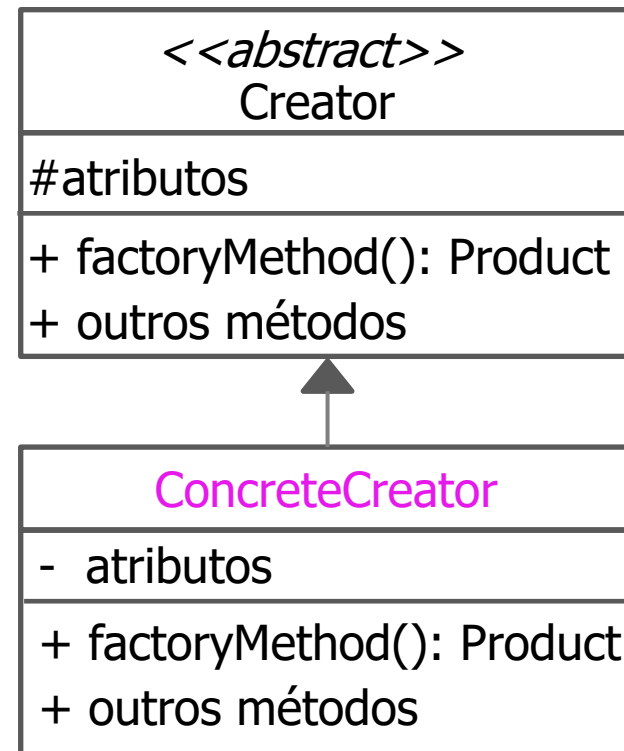
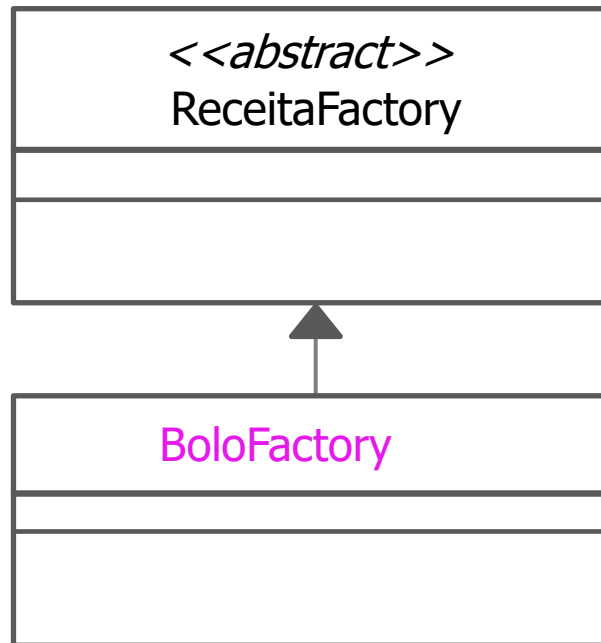
Factory Method - Estrutura



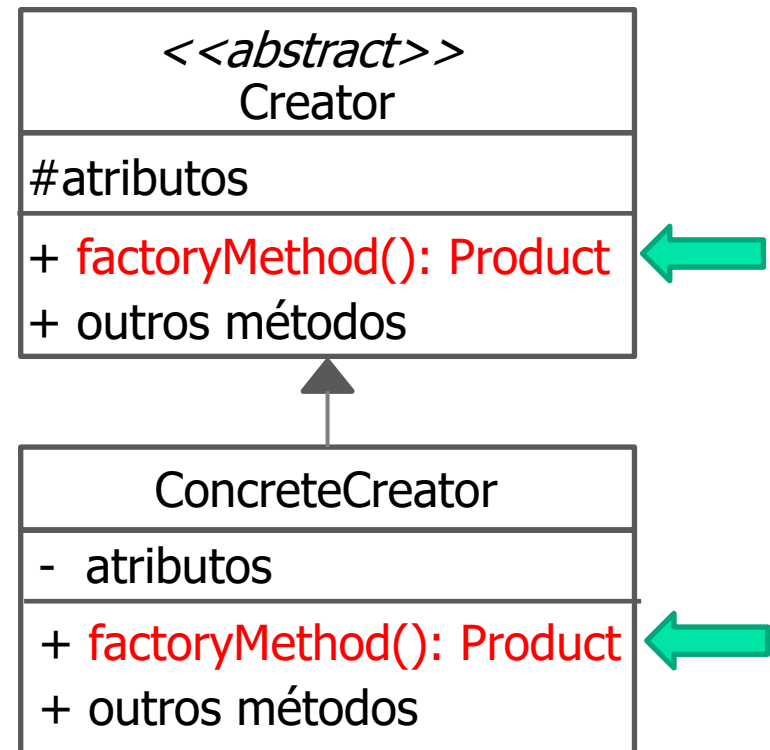
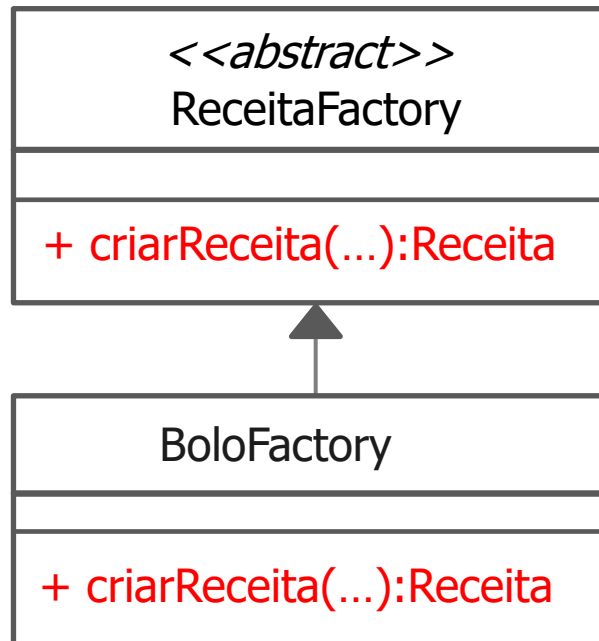
Quem vai ser a classe Creator?



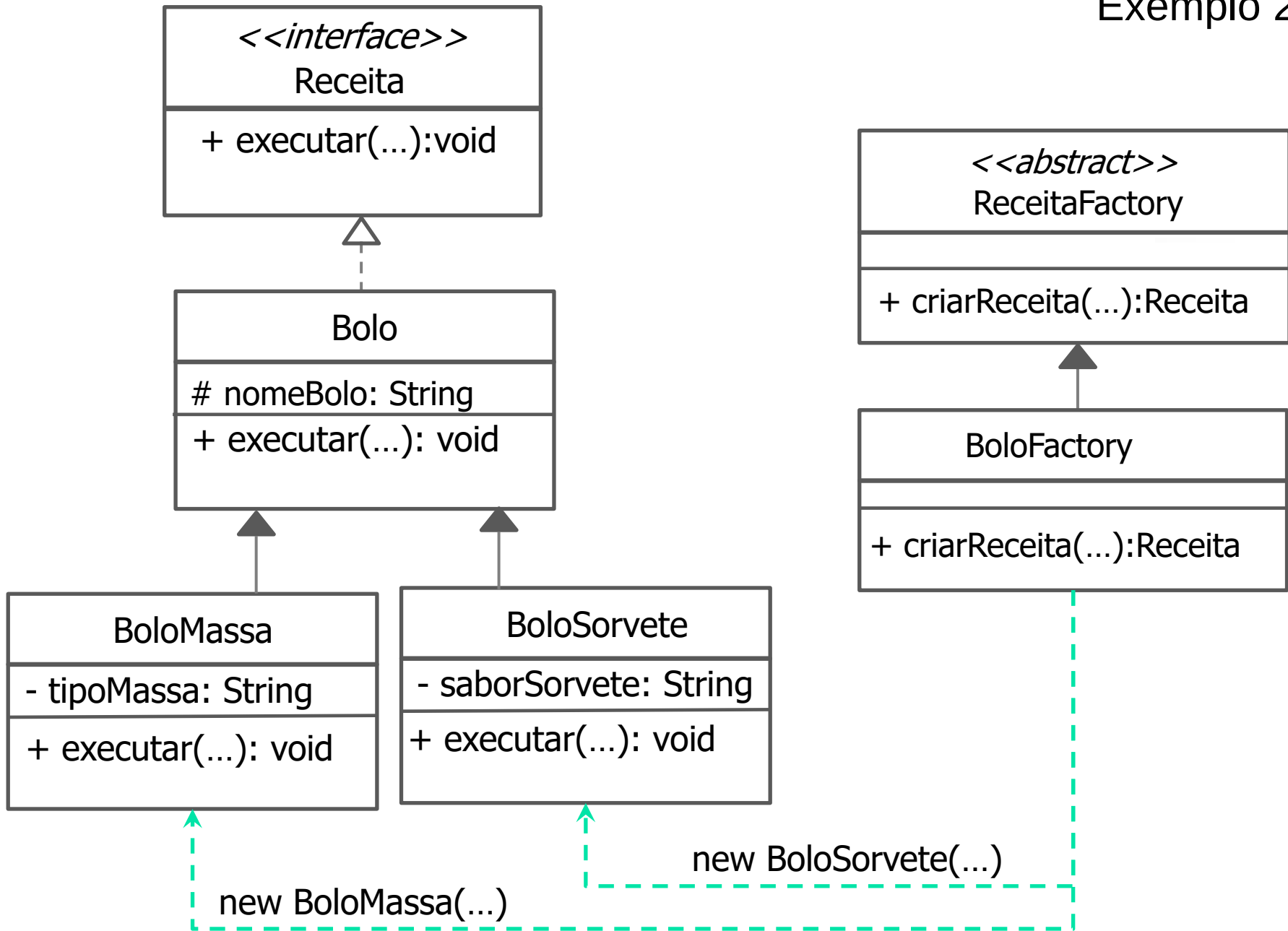
Quem vai ser a classe ConcreteCreator?

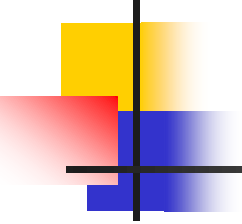


E o factoryMethod()?



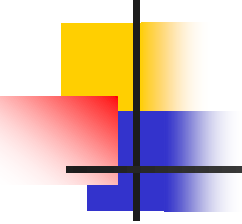
Exemplo 2





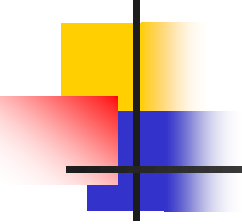
Exercício

- Implemente as classes/interface do Exemplo 2. Verifique que parâmetros podem ser incluídos nos métodos criarReceita(...) e executar(...).



Para praticar...

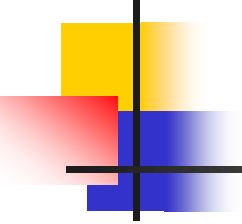
- Pense em uma aplicação em que o padrão Factory Method poderia ser usado. Em seguida, elabore a estrutura para este padrão.



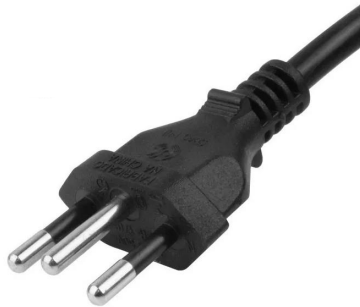
Padrões – Adapter

- Exemplos:

Criação	Estrutural	Comportamental
• Abstract factory • Builder • Factory Method • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Façade • Flyweight • Proxy	• Chain of responsibility • Command • Interpreter • Iterator • Mediator • Memento • Observer • Etc.



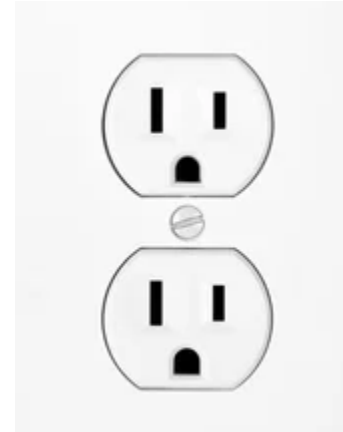
Adapter - Analogia



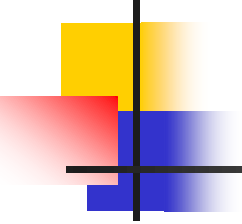
Tomada de notebook



Adaptador

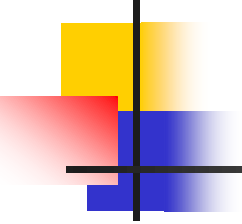


Tomada



Sobre o Adapter

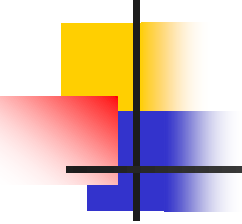
- É um padrão de projeto estrutural (organiza classes e objetos para criar estruturas maiores) também conhecido como *wrapper*.
- Faz exatamente o que um adaptador da vida real faz.
- Evita a dependência com códigos externos.
- Utiliza o conceito de interface.



Sobre o Adapter

- Lista de atributos usadas pelo livro GOF para a descrição dos padrões de projeto:

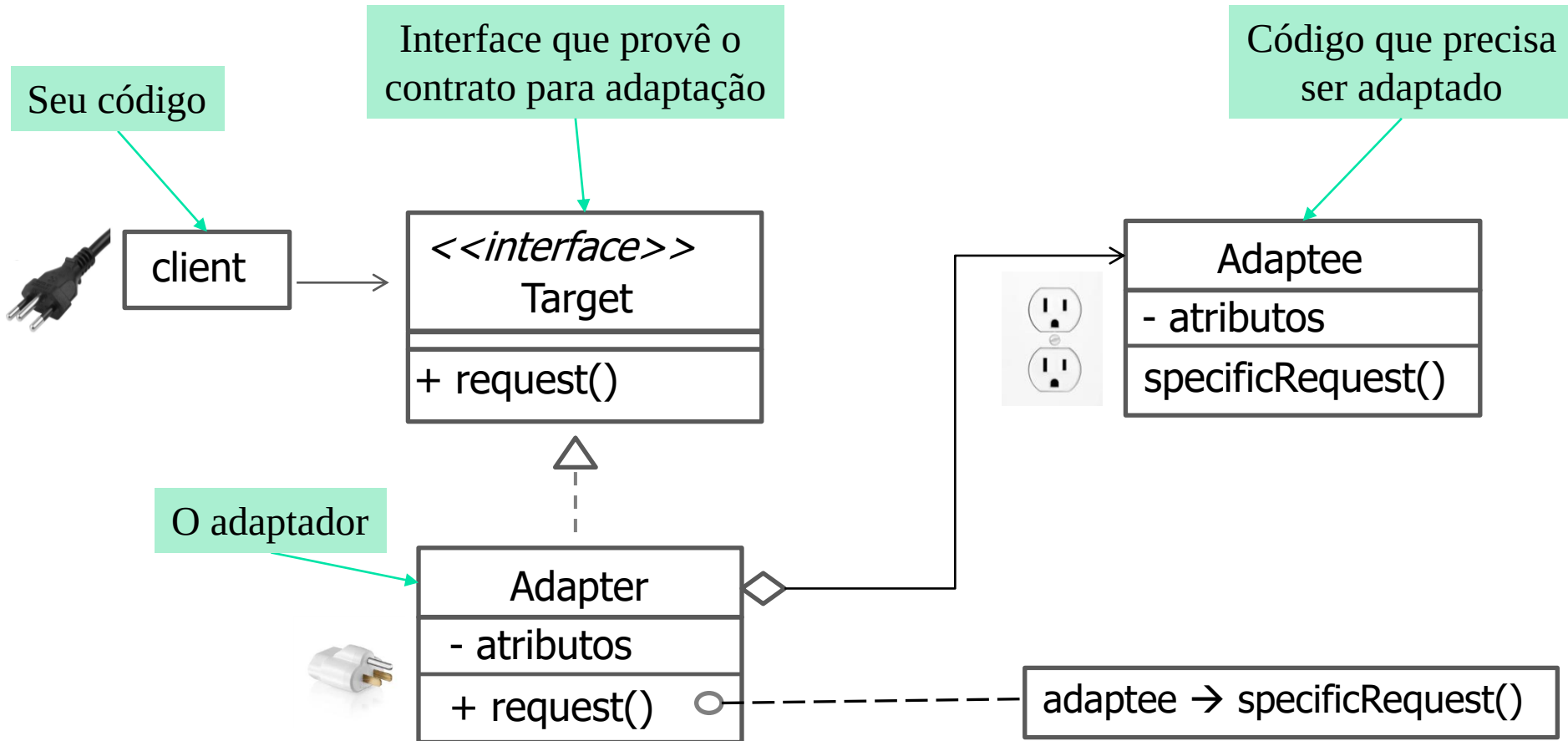
• Nome • Intenção • Motivação • Aplicabilidade • Estrutura • Participantes	• Colaborações • Consequências • Implementação • Exemplo de código • Usos conhecidos • Padrões relacionados
---	--

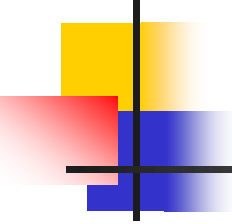


Adapter - Intenção

- Converter a interface de uma classe em outra interface esperada pelos clientes.
- Permite que certas classes trabalhem em conjunto, pois de outra forma seria impossível por causa de suas interfaces incompatíveis.

Adapter - Estrutura



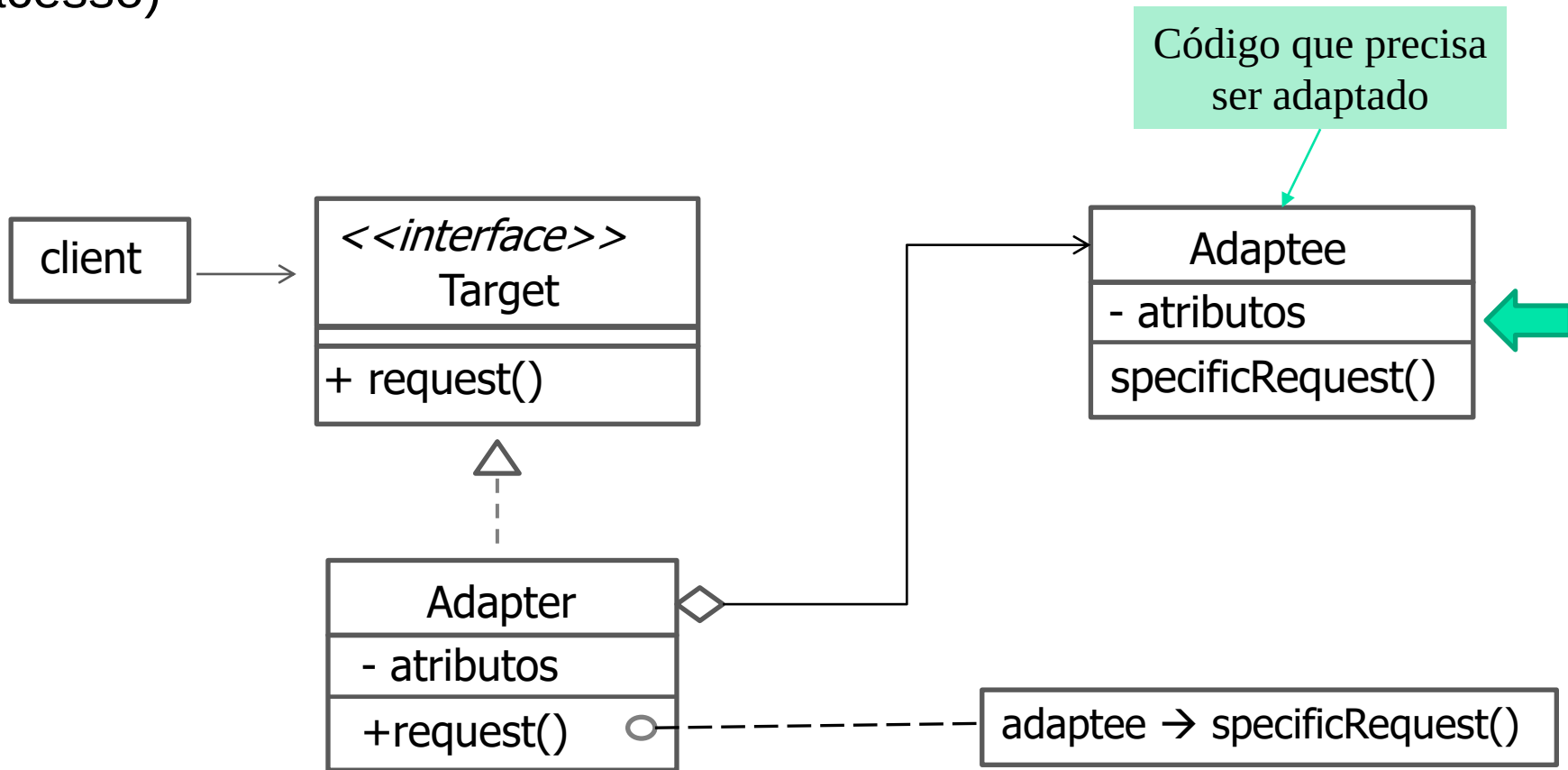


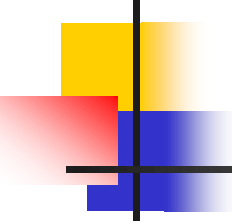
Exemplo: Aplicação de Pagamento

- Desenvolva uma aplicação que ofereça a opção de pagamento no cartão de crédito.

Aplicação - Estrutura

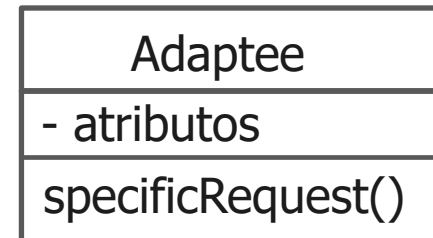
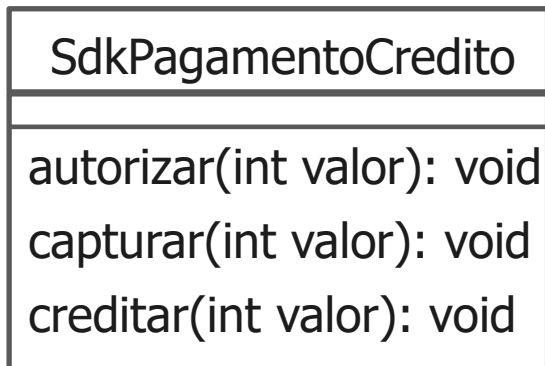
- Classe Adaptee: classe do fornecedor (você não tem acesso)





Exemplo: Aplicação de Pagamento

- Classe do fornecedor (você não tem acesso) chamada de SdkPagamentoCredito.



Código que precisa ser adaptado



```
public class SdkPagamentoCredito
{
    public void autorizar(int valor){
        // reservar o dinheiro
    }

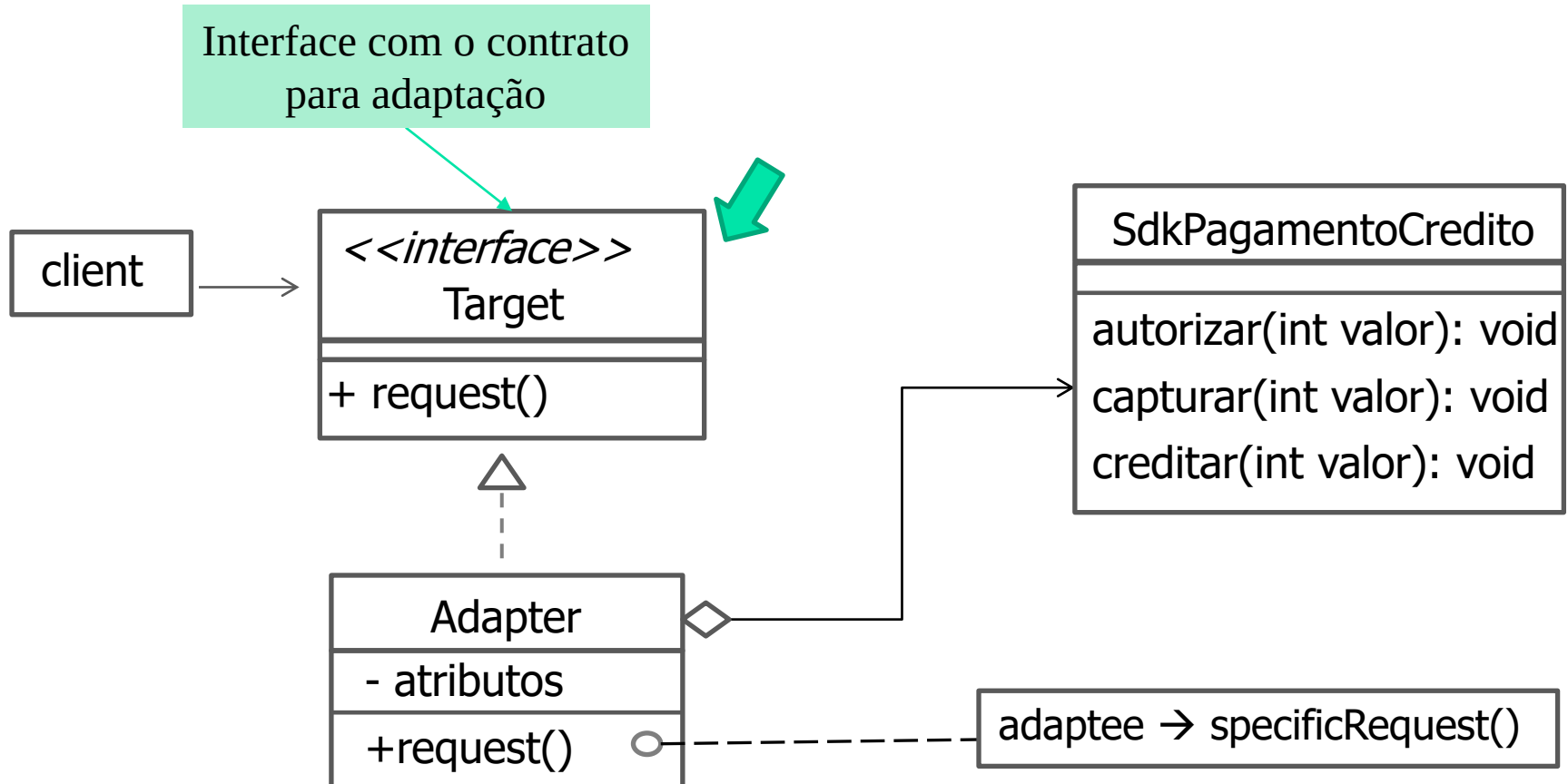
    public void capturar(int valor){
        // efetuar a cobrança
    }

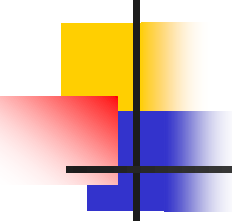
    public void creditar(int valor){
        // extornar dinheiro
    }
}
```

SdkPagamentoCredito
autorizar(int valor): void
capturar(int valor): void
creditar(int valor): void

Aplicação de Pagamento - Adapter

- Interface Target: provê os métodos para adaptação.

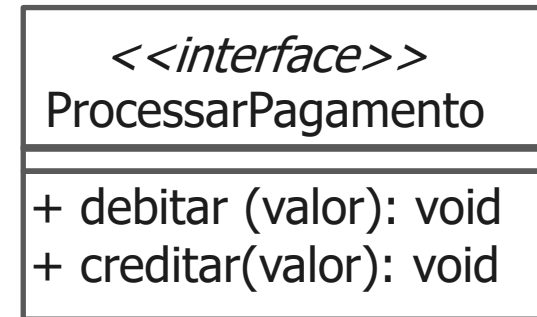
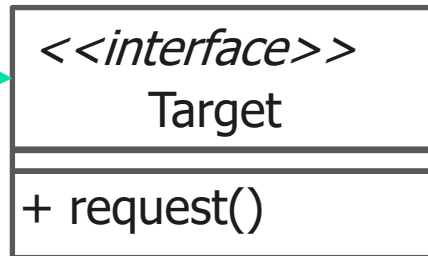




Exemplo: Aplicação de Pagamento

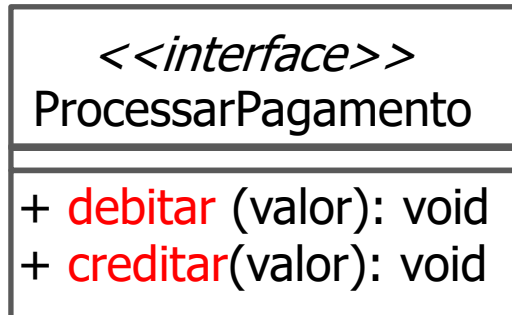
- Interface Target: provê os métodos para adaptação.

Interface que meu
código precisa



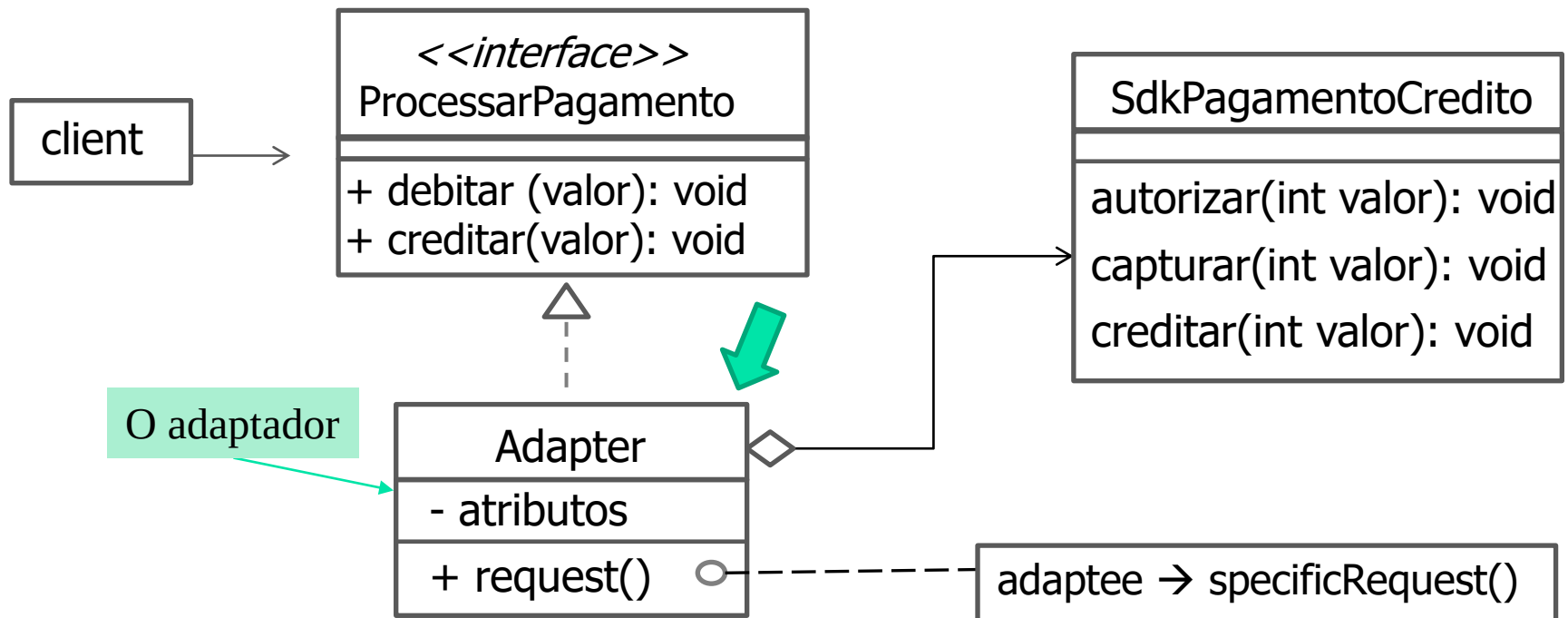
```
public interface ProcessarPagamento
{
    public void debitar(int valor);

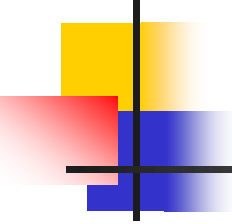
    public void creditar(int valor);
}
```



Aplicação de Pagamento - Adapter

- Classe Adapter: classe interna (você pode alterar, é sua!!!)

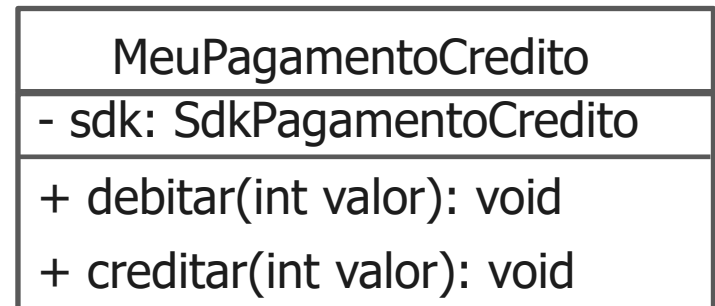
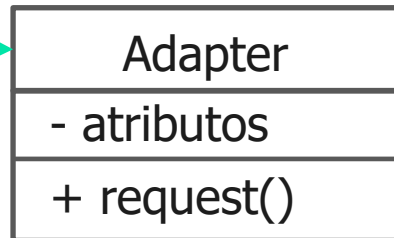




Exemplo: Aplicação de Pagamento

- Classe interna (você pode alterar, é sua!!!)

O adaptador



```

public class MeuPagamentoCredito implements ProcessarPagamento{
    // Atributo do tipo da classe do fornecedor
    private SdkPagamentoCredito sdk;

    public MeuPagamentoCredito(){
        // Instância da classe do fornecedor
        sdk = new SdkPagamentoCredito();
    }

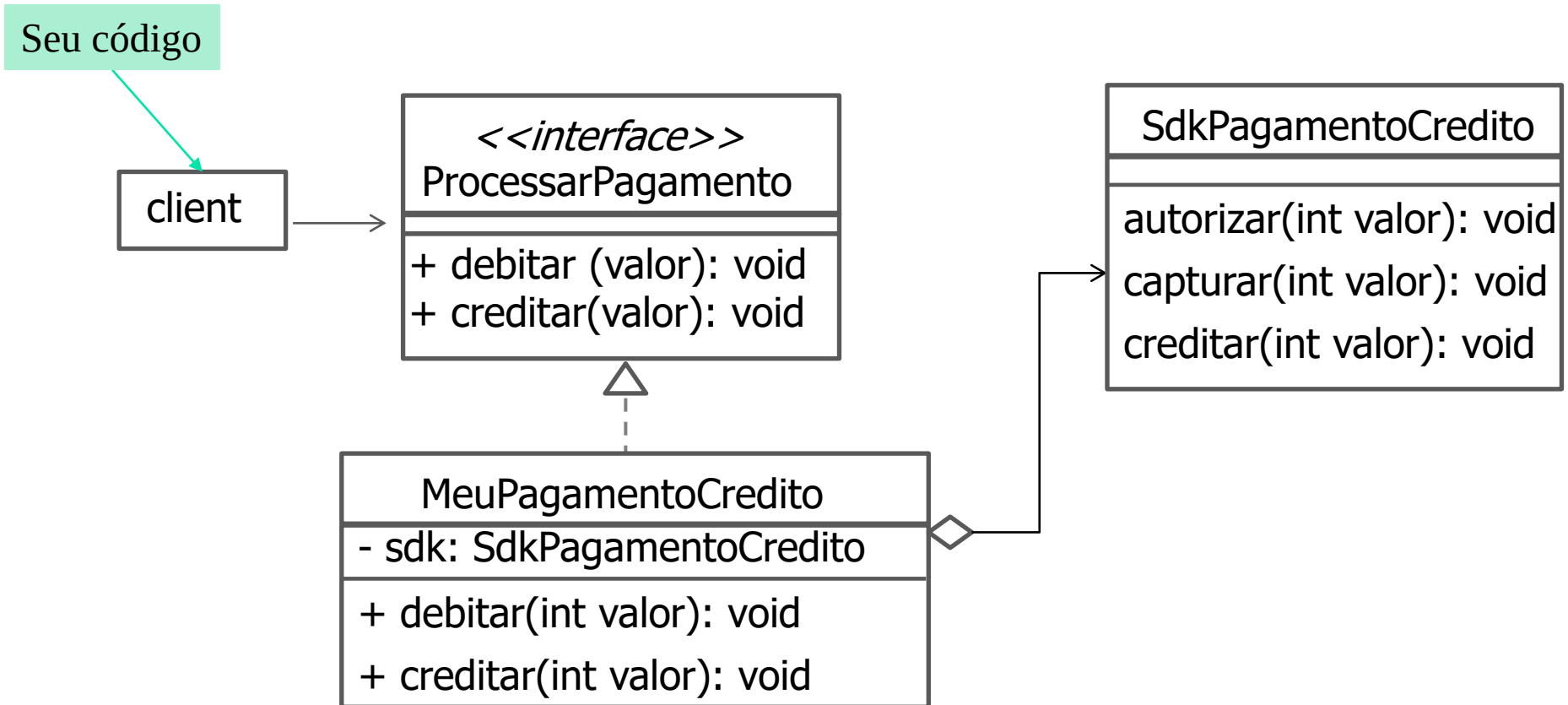
    public void debitar(int valor){
        // Métodos da classe do fornecedor
        sdk.autorizar(valor);
        sdk.capturar(valor);
    }

    public void creditar(int valor){
        // Método da classe do fornecedor
        sdk.creditar(valor);
    }
}

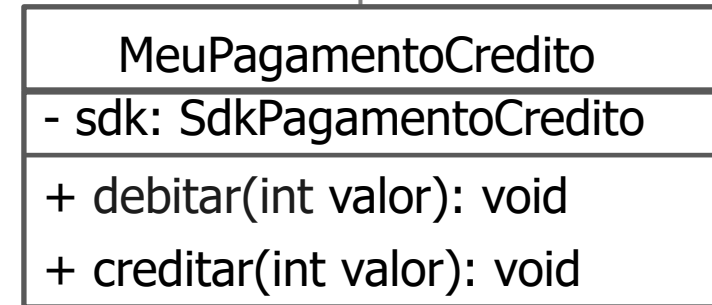
```

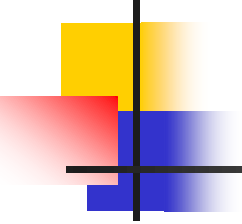
MeuPagamentoCredito
- sdk: SdkPagamentoCredito
+ debitar(int valor): void
+ creditar(int valor): void

Aplicação de Pagamento - Adapter



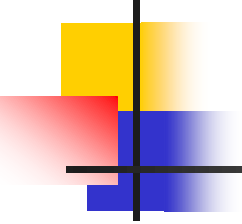
```
public class Principal
{
    public static void main(String []args)
    {
        ProcessarPagamento credito = new MeuPagamentoCredito();
        credito.debitar(240);
    }
}
```





Para praticar...

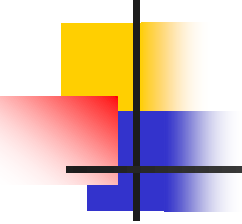
- Pense em uma aplicação em que o padrão Adapter poderia ser usado. Em seguida, elabore a estrutura do padrão.



Padrões - Observer

- Exemplos:

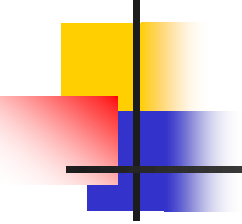
Criação	Estrutural	Comportamental
• Abstract factory • Builder • Factory Method • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Façade • Flyweight • Proxy	• Chain of responsibility • Command • Interpreter • Iterator • Mediator • Memento • Observer • Etc.



Observer

- Lista de atributos usadas pelo livro GOF para a descrição dos padrões de projeto:

• Nome • Intenção • Motivação • Aplicabilidade • Estrutura • Participantes	• Colaborações • Consequências • Implementação • Exemplo de código • Usos conhecidos • Padrões relacionados
---	--

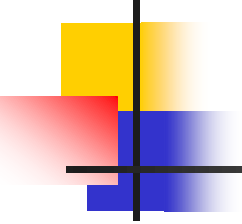


Observer - Intenção

- Define uma dependência um para muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são automaticamente notificados e atualizados.

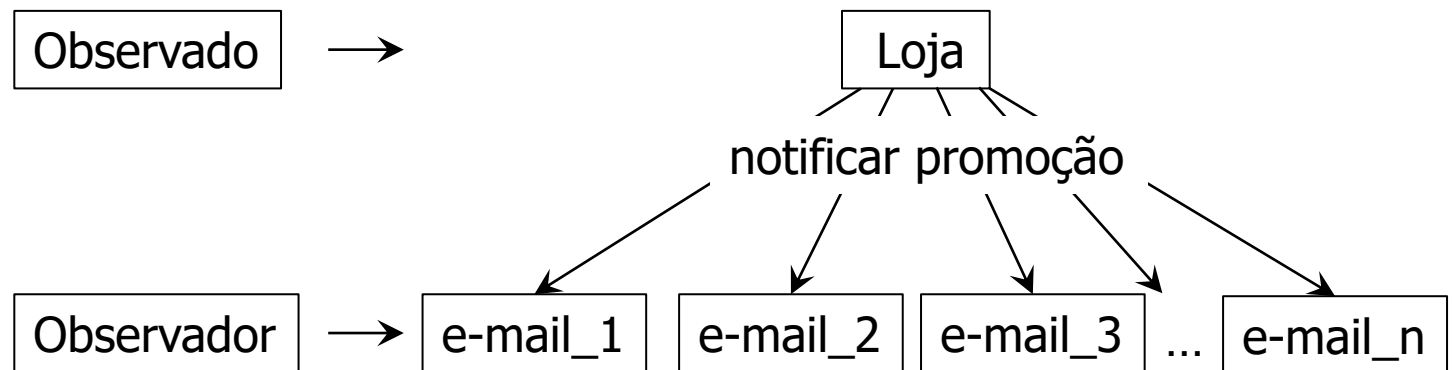
Objeto observado Métodos: adicionar, remover, notificar

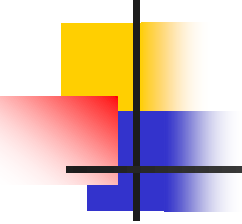
Observador Observador Observador ... Observador → dependentes



Padrão Observer - Ideia

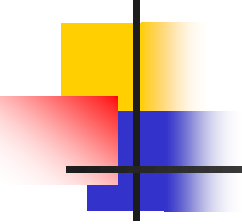
- Funciona como uma *newsletter* (e-mail informativo) de um site.





Sobre o Observer

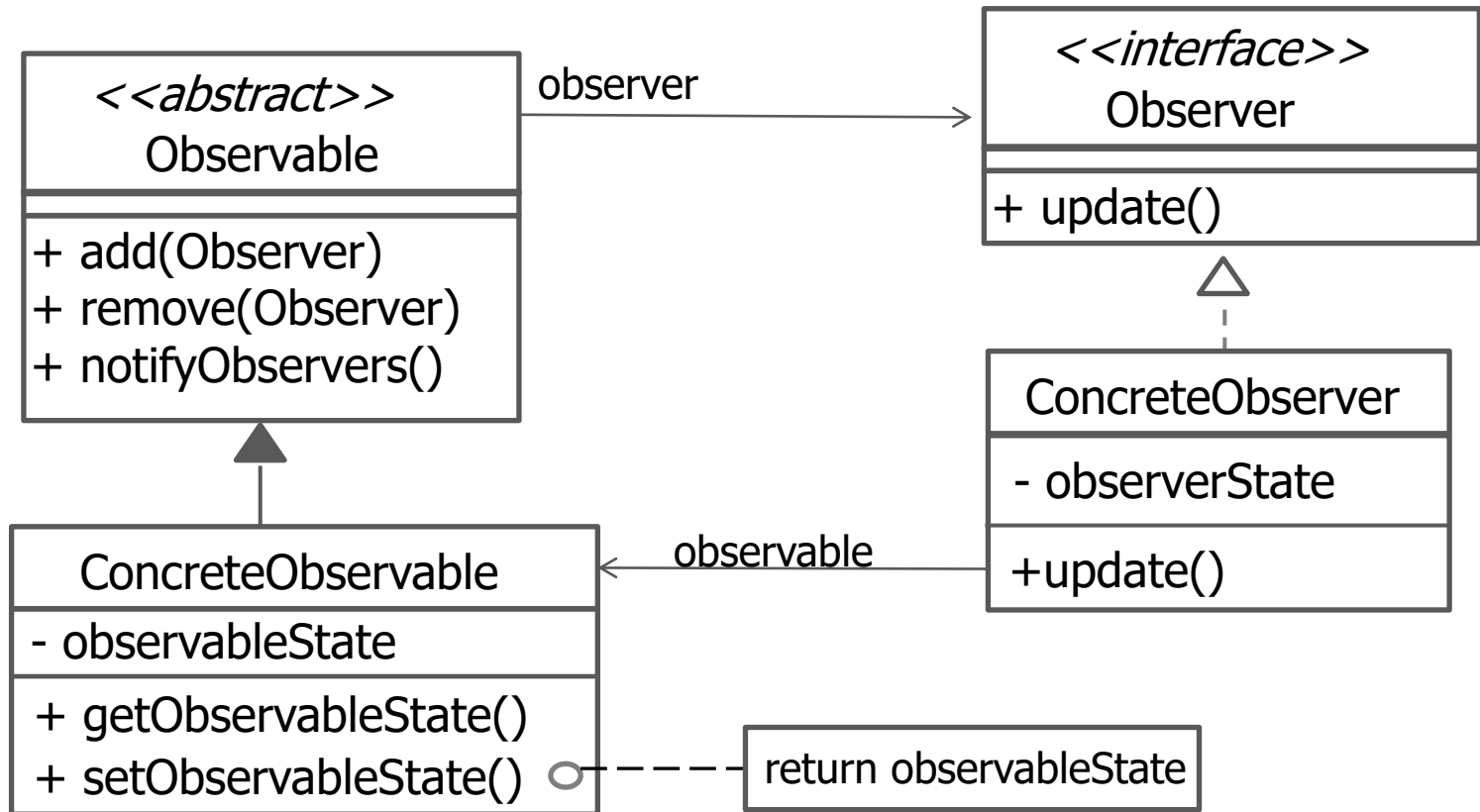
- É um padrão de projeto comportamental (classes e objetos interagem e distribuem responsabilidades na aplicação) .
- Implementados com dois tipos de objetos: objetos observados (*observable*) e objetos observadores (*observer*).
- Objetos observados (*observable*) possuem referência para todos os seus observadores (*observer*).

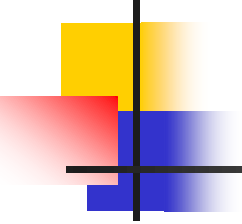


Sobre o Observer

- Objetos observados (*observable*) podem adicionar, remover e notificar todos os observadores (*observer*) quando seu estado muda.
- Objetos observadores (*observer*) devem ter meios para receber as notificações de seu observado (*observable*). Normalmente, isso é feito por meio de um método.
- Utiliza o conceito de interface e classe abstrata.

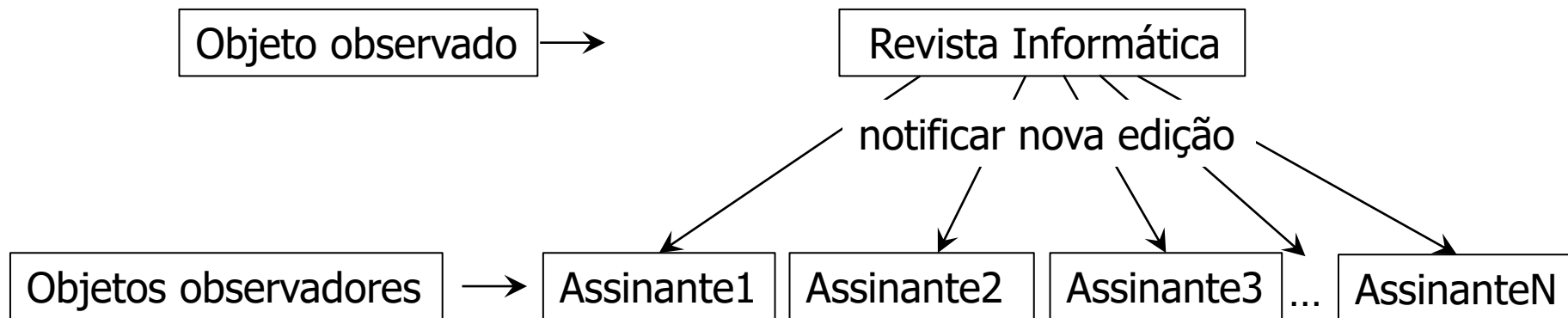
Observer - Estrutura



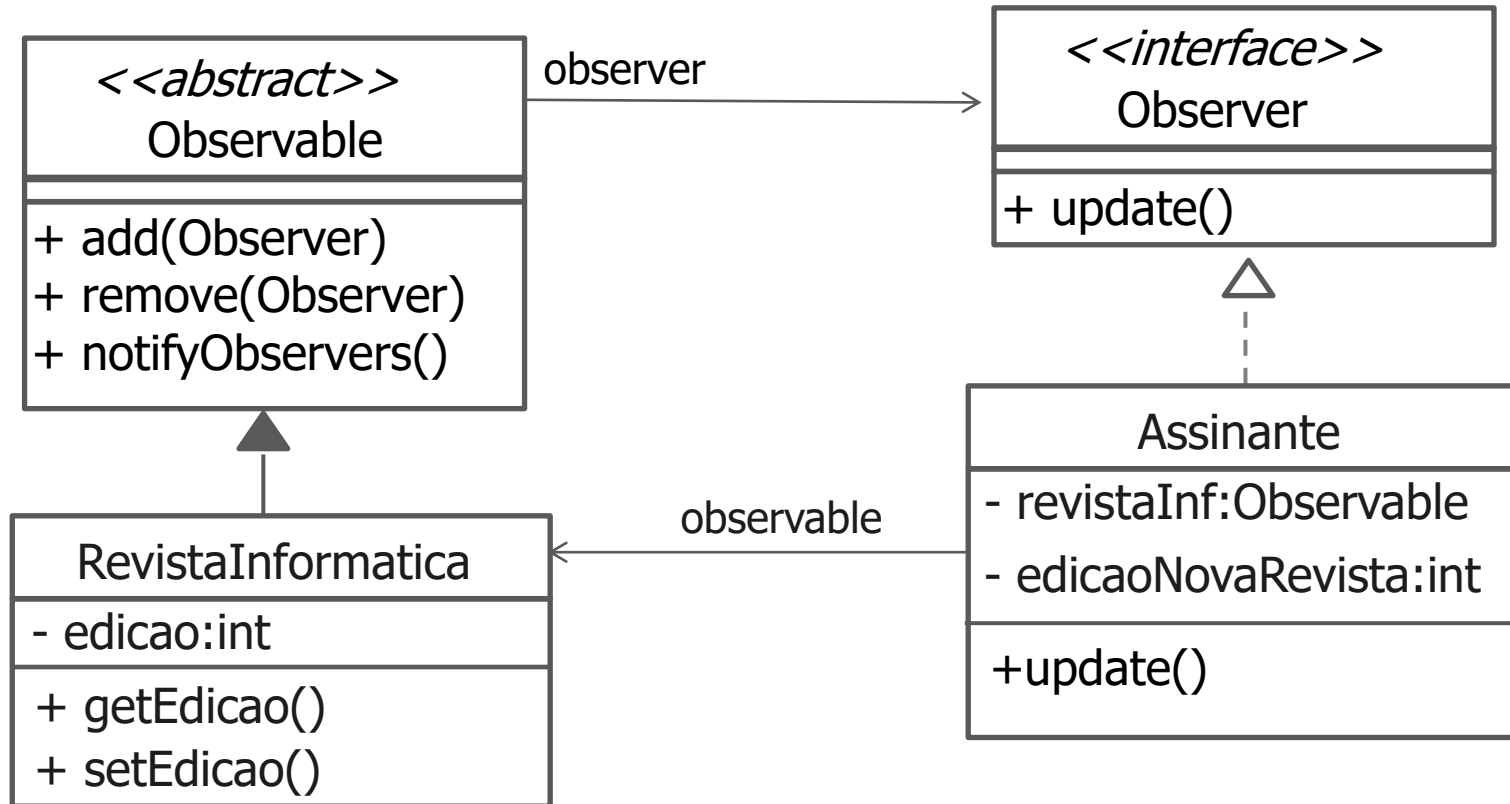


Exemplo: Revista Informática

- Aplicação que notifica os assinantes de novas edições da Revista de Informática .



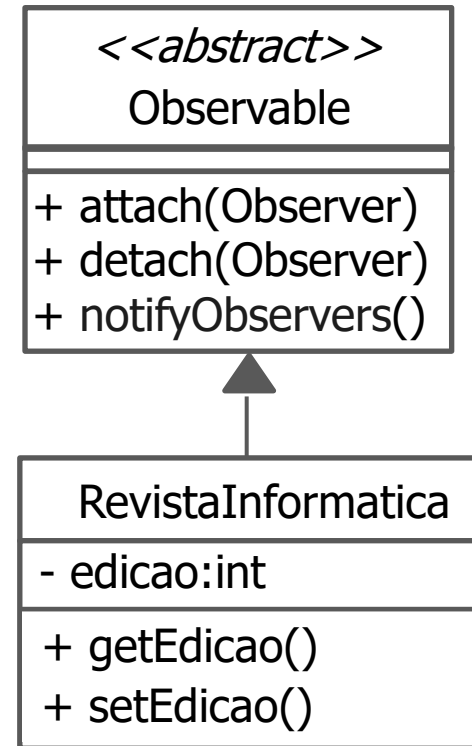
Revista Informática - Observer



```
public class RevistaInformatica extends Observable
{
    private int edicao;

    public int getEdicao(){
        return this.edicao;
    }

    public void setEdicao (int novaEdicao){
        if(novaEdicao > 0)
        {
            this.edicao = novaEdicao;
            /* chamada do método para
            notificar os observadores */
            notifyObservers();
        }
    }
}
```



```
public class Assinante implements Observer
```

```
{
```

```
    private Observable revistaInf;
```

```
    private int edicaoNovaRevista;
```

```
    public Assinante(Observable revistaInfo){
```

```
        this.setRevistaInf(revistaInfo);
```

```
        revistaInf.add(this);
```

```
    }
```

```
// Métodos get/set
```

```
    public void update(Observable revistaInfo)
```

```
    {
        if(revistaInfo instanceof RevistaInformatica){
```

```
            RevistaInformatica rev = (RevistaInformatica) revistaInfo;
```

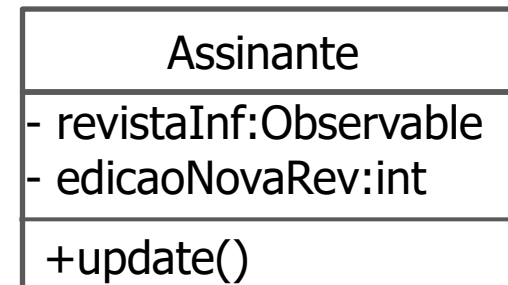
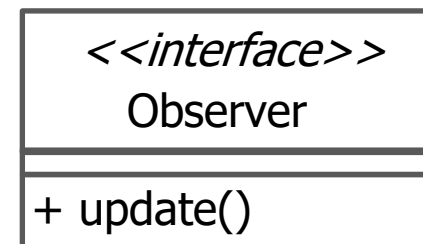
```
            this.edicaoNovaRevista = rev.getEdicao();
```

```
            S.o.p("Atenção! Edição " + edicaoNovaRevista + " disponível");
```

```
        }
```

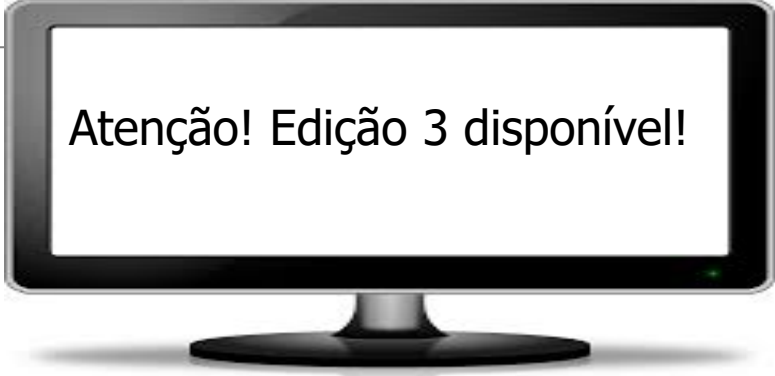
```
    }
```

```
}
```

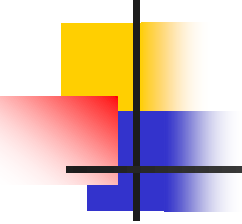


```
public class Principal
{
    public static void main(String []args)
    {
        int novaEdicao = 3;
        RevistaInformatica revista = new RevistaInformatica();
        Assinante assinante = new Assinante(revista);

        revista.setNovaEdicao(novaEdicao);
    }
}
```

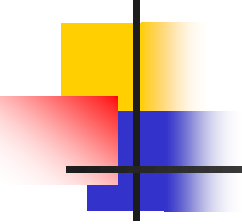
A black computer monitor with a silver stand, displaying a white screen with black text. The text reads "Atenção! Edição 3 disponível!".

Atenção! Edição 3 disponível!



Para praticar...

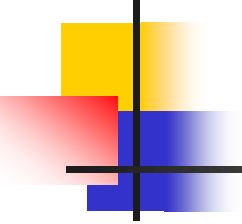
- Pense em uma aplicação em que o padrão Observer poderia ser usado. Em seguida, elabore a estrutura do padrão.



Exemplo 1

- Que padrão de projeto utilizar?

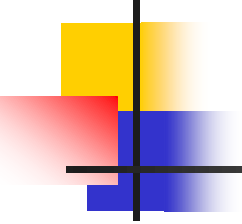
Desenvolva uma aplicação para rede sociais que sinalize quando uma nova postagem é exibida no feed dos seus usuários. O feed é o local onde os usuários interagem e se mantêm atualizados com o conteúdo compartilhado por seus amigos e seguidores.



Exemplo 2

- Que padrão de projeto utilizar?

Desenvolva uma aplicação para uma empresa de dispositivos eletrônicos personalizados. Essa empresa produz smarthpones, tablets e laptops, cada um com suas especificações. A empresa precisa de um mecanismo que facilite a produção desses dispositivos e garanta que as características solicitadas pelos clientes sejam incorporadas.



Exemplo 3

- Que padrão de projeto utilizar?

Desenvolva uma aplicação de streaming de música que permita que o usuário tenha acesso a diferentes serviços de música, como Spotify, Apple Music e YouTube Music usando uma única interface. Um streaming de música consiste na transmissão contínua de música pela Internet, sem que seja necessário baixar a música localmente.

1 - Desenvolva uma aplicação para rede sociais que sinalize quando uma nova postagem é exibida no *feed* dos seus usuários.

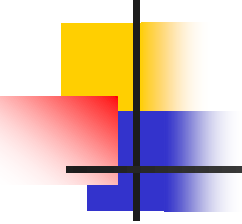
Resp: Observer

2 - Desenvolva uma aplicação para uma empresa de dispositivos eletrônicos personalizados. Essa empresa produz *smarthphones*, *tablets* e *laptops*, cada um com suas especificações. A empresa precisa de um mecanismo que facilite a produção desses dispositivos e garanta que as características solicitadas pelos clientes sejam incorporadas.

Resp: Factory Method

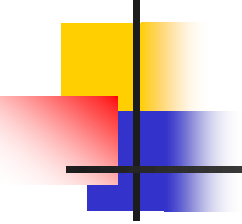
3 - Desenvolva uma aplicação de streaming de música que permita que o usuário tenha acesso a diferentes serviços de música, como Spotify, Apple Music e YouTube Music usando uma única interface.

Resp: Adapter



Para praticar...

- Monte a estrutura para os padrões de projeto mostrados nos Exemplos 1, 2 e 3.



Referência

- Gamma, E., Helm, R., Johnson, R., Vlissides, J. (2007). Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre, RS: Bookman.